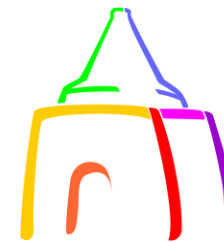


Control de Servidores

GNU/Linux básico y Windows



IES
Puerto
de la
Cruz
Telesforo Bravo

Directorios en GNU/Linux

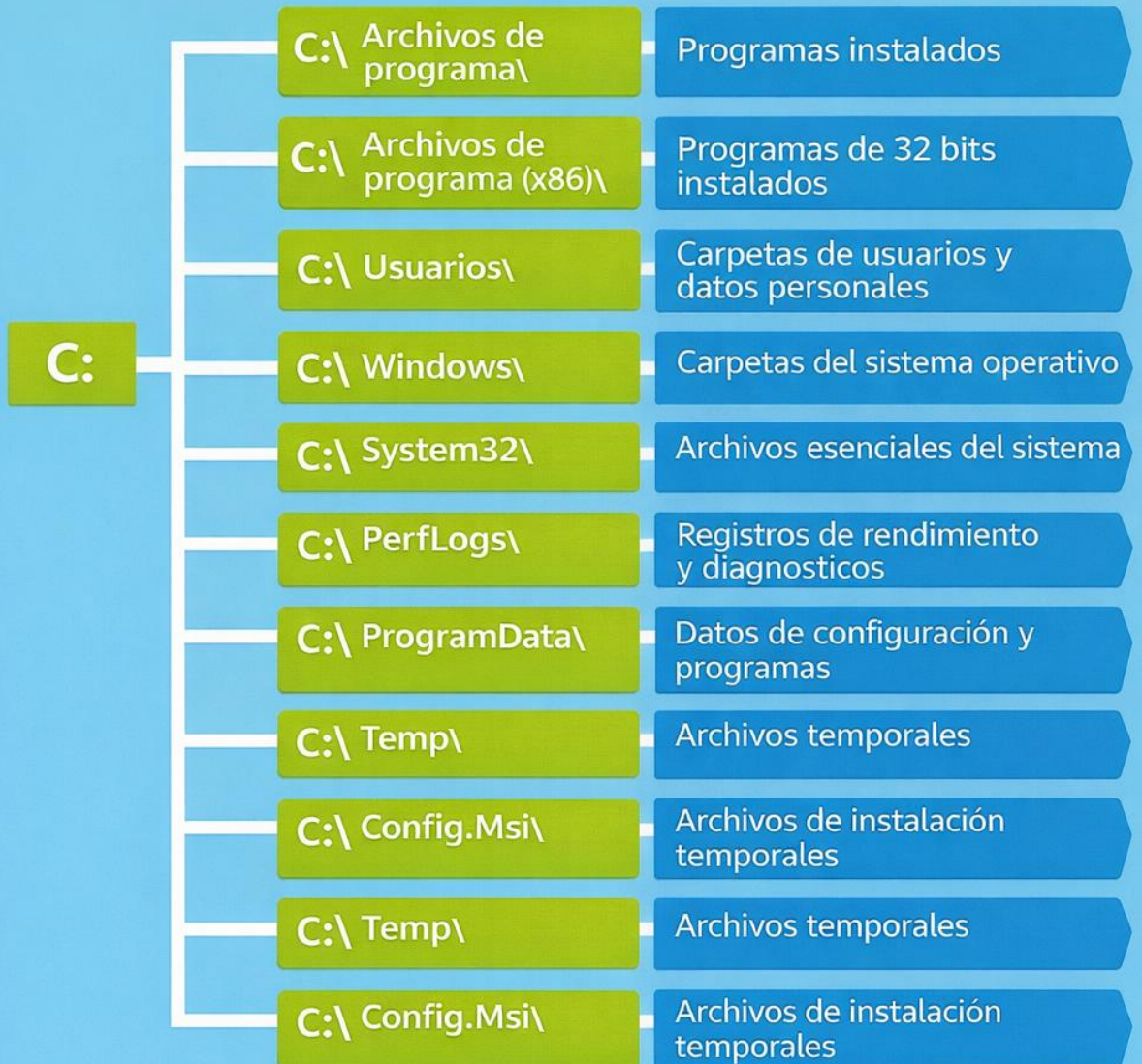
OJO: Hoy en día tanto **/bin** como **/sbin** suelen ser enlaces *simbólicos* a **/usr/bin** y **/usr/sbin** respectivamente.



Directorios en Windows



Estructura de Directorios en Windows 10-11



Comandos Linux básicos

Comandos de Ayuda

- **comando --help** → ayuda de párams. de forma resumida
- **man comando** → ver el manual completo del comando
- **help comando** → ayuda comandos internos de Shell
- **whatis comando** → descripción corta del comando

Comandos Linux básicos

Comandos útiles de ayuda

- **which comando** → donde está el ejecutable/comando.
- **type comando** → Qué es el comando realmente.
- **file archivo** → qué tipo de archivo es, qué formato.
- **command -v comando** → Qué se ejecutará realmente.
- **clear** → Borrar la pantalla, no afecta a nada.

Comandos Linux básicos

Gestión de directorios y archivos

- Listar directorios y archivos

`ls, ls -l, ls -la`

- Cambiar de directorio

`cd`

- Crear directorios

`mkdir, mkdir -p`

- Borrar directorios

`rmdir, rm -rf`

- Borrar archivos

`rm, rm -f, rm -rf`



Comandos Linux básicos

Gestión de directorios y archivos

Directorios básicos que deben conocer:

- `.` → el punto siempre hace referencia al directorio actual.
- `cd .` → (no se mueve de donde está)
- `..` → directorio anterior
- `cd ..` → nos vamos a la carpeta inmediatamente inferior.
- `/` → directorio raíz o principal del sistema, el primero.
- `cd /` → vamos a la raíz del sistema.
- `~` → directorio del usuario (logueado), el home del usuario. En Linux normalmente: `/home/usuario`
- `cd ~` → nos vamos a la carpeta del usuario
- `cd ~/Descargas` → nos vamos a la carpeta *Descargas* del usuario sin necesidad de poner `cd /home/user/Descargas`
- `cd -` → volver al directorio anterior.

Comandos Linux básicos

Gestión de directorios y archivos

- Saber en qué directorio estoy *pwd* (path working directory):

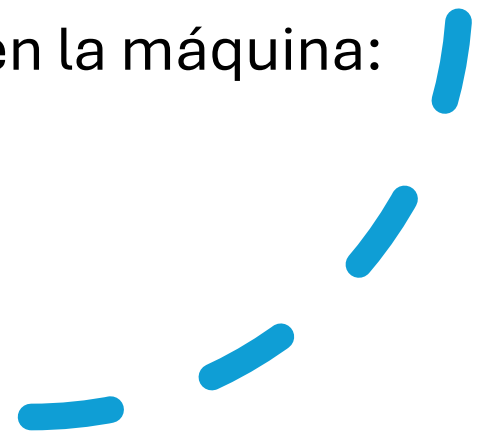
pwd

- Saber el usuario que está logueado en la máquina.

whoami

- Saber todos los usuarios logueados en la máquina:

who



Comandos Linux básicos

Gestión de directorios y archivos

- Imprimir el contenido de un archivo en pantalla:

cat archivo

- Ordenar un archivo por líneas:

sort archivo

- Mostrar solo las primeras líneas de un archivo:

head archivo

- Mostrar sólo las últimas líneas de un archivo:

tail archivo

- Contar líneas, palabras, bytes de un archivo:

wc archivo, wc -l archivo (-l cuenta líneas)

- Ordenar *archivo* y además mostrar resultados únicos:

sort archivo | uniq

Linux filesystem explicación

Gestión de directorios y archivos

- Crea un archivo (vacío) si no existe o modifica sólo su fecha de última modificación si ya existe:

touch archivo

**OJO: Hay gente que piensa que es lo mismo que:
>Archivo**

Y no! “>” es un redireccionador! y puesto así crea el archivo si no existe, PERO se lo carga (vacia su contenido) si existiera. Para añadir se usa >> archivo. Pero tampoco se recomienda en este contexto.

Comandos Linux básicos

Variable de entorno importantes

- **\$HOME** → directorio personal del usuario.
 - **\$PATH** → todas las rutas que tiene el usuario actual para buscar comandos y programas ejecutables.
 - **\$USER** → nombre del usuario actual.
 - **\$PWD** → Como *pwd*, directorio de trabajo actual.
 - **\$SHELL** → La shell que estamos usando
 - **\$?** → salida del último comando ejecutado.
 - **\$\$** → PID del proceso actual (la shell).
-
- **printenv** → Imprime todas las variables de entorno.
-
- **variable=valor** → Crea una variable local de nombre **variable** con valor, **valor**.
 - **export variable** → Convierte variable local en variable de entorno.

Permisos y usuarios

Tipo
de
archivo

Permisos
del
propietario

Permisos
del
grupo

Permisos
del
resto

-	rw	-	rw	-	r	-	-
---	----	---	----	---	---	---	---



Permisos y usuarios

- Cambiar permisos de archivos:

chmod

- Cambiar el propietario y/o grupo de archivos:

chown

- Cambiar solo el grupo del archivo

chgrp

Permisos y usuarios

formato Octal

- Permisos en formato OCTAL:

4 → Lectura → r (Read)

2 → Escritura → w (Write)

1 → Ejecución → x (eXecution)

La suma de todos es 7, que equivale a permisos totales.

777 = rwxrwxrwx

755 = rwxr-xr-x

644 = rw-r--r--

600 = rw-----

Permisos y usuarios

ejemplos
formato
octal

chmod 777 archivo.txt	# rwxrwxrwx -> lectura, escritura y ejecución para todos
chmod 755 script.sh	# rwxr-xr-x -> control total para el usuario, lectura y ejec. para grupo y otros
chmod 700 privado.sh	# rwx----- -> solo el usuario puede leer, escribir y ejecutar
chmod 644 archivo.txt	# rw-r--r-- -> lectura/escritura para usuario, solo lectura para grupo y otros
chmod 600 claves.txt	# rw----- -> solo el usuario puede leer y escribir
chmod 666 notas.txt	# rw-rw-rw- -> lectura y escritura para todos, nadie puede ejecutar
chmod 444 lectura.txt	# r--r--r-- -> solo lectura para todos
chmod 400 secreto.txt	# r----- -> solo lectura para el usuario
chmod 775 carpeta	# rwxrwxr-x -> control total para usuario y grupo, lectura y ejec. para otros
chmod 711 script.sh	# rwx--x--x -> el usuario tiene todo, grupo y otros solo pueden ejecutar
chmod 754 programa.sh	# rwxr-xr-- -> usuario todo, grupo lectura y ejecución, otros solo lectura
chmod 640 informe.txt	# rw-r----- -> usuario lectura/escritura, grupo solo lectura, otros nada
chmod 660 trabajo.txt	# rw-rw---- -> usuario y grupo lectura/escritura, otros nada
chmod 555 script.sh	# r-xr-xr-x -> todos pueden leer y ejecutar, nadie escribir
chmod 500 script.sh	# r-x----- -> solo el usuario puede leer y ejecutar
chmod 000 bloqueado.txt	# ----- -> nadie tiene permisos



**Permisos y
usuarios**

ejemplos
formato
simbólico

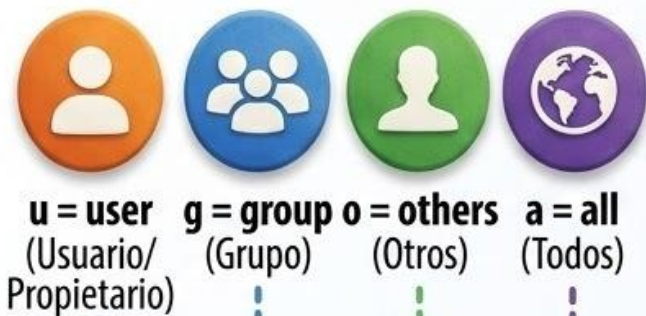
Ver siguiente página

**No entra en el examen,
pero sí en la “Vida”.**



Guía de Permisos Simbólicos en Linux (chmod) para 1º DAM

QUIÉN (WHO): 



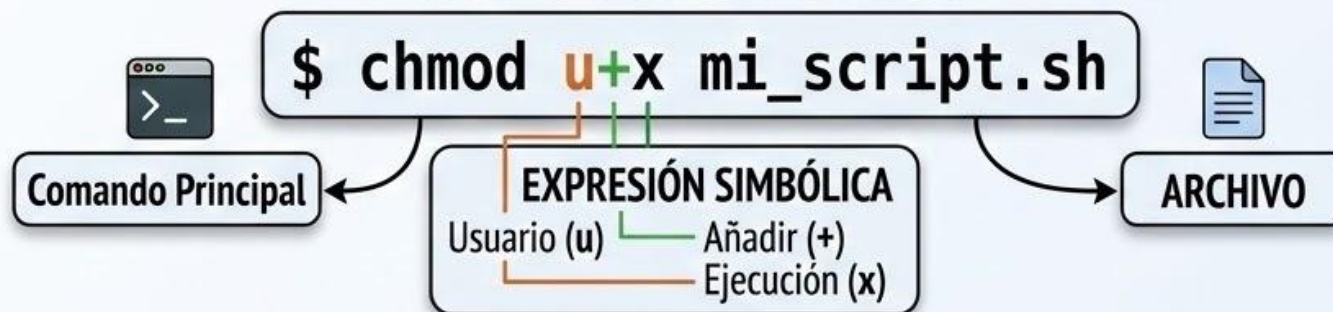
QUÉ ACCIÓN (ACTION): 



QUÉ PERMISO (PERMISSION): 



CÓMO ARMAR EL COMANDO

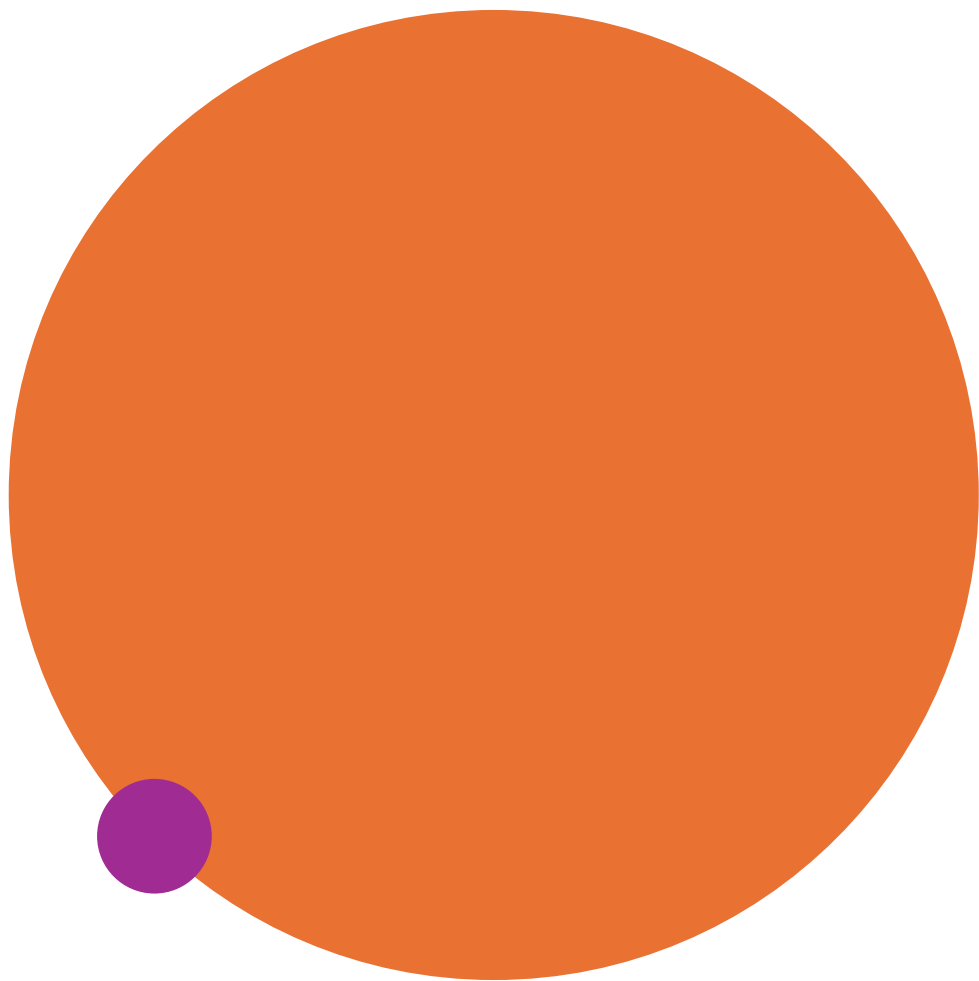


EFFECTO: u+x -> El Usuario Propietario (u) ahora tiene permisos para Ejecutar (x) el archivo "mi_script.sh"

Permisos y usuarios

ejemplos
formato
simbólico

<code>chmod a+x script.sh</code>	<code># añade permiso de ejecución a todos.</code>
<code>chmod g+w archivo.txt</code>	<code># añade permiso de escritura al grupo</code>
<code>chmod o-r privado.txt</code>	<code># quita permiso de lectura a otros</code>
<code>chmod a+r manual.txt</code>	<code># añade permiso de lectura a todos</code>
<code>chmod u-w notas.txt</code>	<code># quita permiso de escritura al usuario</code>
<code>chmod g-x carpeta</code>	<code># quita permiso de ejecución al grupo</code>
<code>chmod o+x script.sh</code>	<code># añade permiso de ejecución a otros</code>
<code>chmod u=rw archivo.tx t</code>	<code># deja al usuario solo lectura y escritura</code>
<code>chmod g=r informe.txt</code>	<code># deja al grupo solo lectura</code>
<code>chmod o= archivo.txt</code>	<code># quita todos los permisos a otros</code>
<code>chmod a=rw archivo.txt</code>	<code># deja lectura y escritura para todos, sin ejecución</code>
<code>chmod u=rwx,g=rx,o=r app.sh</code>	<code># define exactamente los permisos para cada grupo</code>
<code>chmod go-rwx secreto.txt</code>	<code># quita lectura, escritura y ejecución a grupo y otros</code>
<code>chmod ugo+x script.sh</code>	<code># añade ejecución a usuario, grupo y otros</code>
<code>chmod a-x archivo.txt</code>	<code># quita ejecución a todos</code>



**A partir de aquí sí entra
En el examen.
Y en la vida 😊**



Usuarios y grupos



Normalmente, cuando hablamos de privilegios elevados en GNU/Linux, nos referimos a actuar con permisos de root, directamente o mediante sudo



Usuarios y grupos

- Gestión de usuarios y grupos (requieren sudo o root):

chown
chgrp*
useradd
usermod
passwd*
deluser
groupadd
sudo
visudo

- Comandos útiles que no requieren sudo/root:

id
whoami
groups
newgrp
passwd

** chgrp: se puede usar sin privilegios si el usuario es el propietario del archivo y ya pertenecer al grupo destino.*

** passwd: se puede usar sin privilegios para cambiar tu propia password en el sistema.*

Comandos básicos de ayuda

Paginadores en Linux

Son comandos que nos sirven para poder ver por páginas el resultado de comandos cuando el resultado es demasiado extenso (ocupa más de una pantalla):

- **more**
comando | more
- **less**
comando | less

En contra de la lógica **less** es más potente.



Memoria y disco

- **Memoria RAM y SWAP (memoria total, memoria, libre, usada, etc.):**

`free, free -h`

- **Espacio en disco por particiones y fs (total, libre, usada):**

`df`

`df -h`

`df -hT`

- **Espacio usado por directorios/archivos:**

`du -sh dir`

Con `-h` → “**h**uman-mode”. Le pedimos las unidades en formato “humano” en vez de en bytes.

Con `-s` → “**s**ummarize”. Muestra solo el resumen no archivo por archivo. Aplica solo con el comando **du**.

Gestión de Memoria – Linux vs Windows

Comparativa: Windows vs Linux

Característica	Windows 10/11	Linux
Estrategia	Paginación agresiva y compresión.	Matar al proceso más pesado (OOM Killer).
Comportamiento	El sistema se vuelve lento/congelado.	El sistema sigue fluido pero se cierra una app.
Aviso al usuario	Ventana emergente de "Poca memoria".	Normalmente nada (puedes verlo en <code>dmesg</code>).
Resultado final	El programa falla al pedir más RAM.	El Kernel envía un <code>SIGKILL</code> .

Pipes y redirecciones

- **Redirecciones** `>`, `>>`, `<`, `2>`, `2>&1`, `&>`
Controlan a dónde va *stdout*, *stderr* y de dónde viene *stdin*.
- **Pipes:** Encadena comandos, la salida (*stdout*) de uno pasa a ser la entrada (*stdin*) del siguiente

```
ls -l | wc -l
```

```
cat fichero | grep "user" | wc -l
```

NOTA: Tienen un .pdf en el aula donde se explican detalladamente los Pipes y las redirecciones.

Operadores lógicos && y || de comandos

- && y || son **operadores lógicos** aplicados a la **salida (código de retorno)** del comando anterior.

Comando 1 → OPERADOR → Comando 2

&& → Comando 2 solo si Comando 1 tuvo éxito.

|| → Comando 2 solo si Comando 1 falló.

mkdir /prueba || echo "La creación del directorio falló"

mkdir "\$HOME/Descargas/prueba" 2>/dev/null && echo "OK" || echo "ERROR" >&2

NOTA: Tienen un .pdf en el aula donde se explican detalladamente los Pipes y las redirecciones.

Enlaces linux

- Enlaces simbólicos (los más usados)

`ln -s /ruta/archivo enlace`

- Enlaces duros o hard links (realmente apuntan a inodo)

`ln /ruta/archivo enlace`

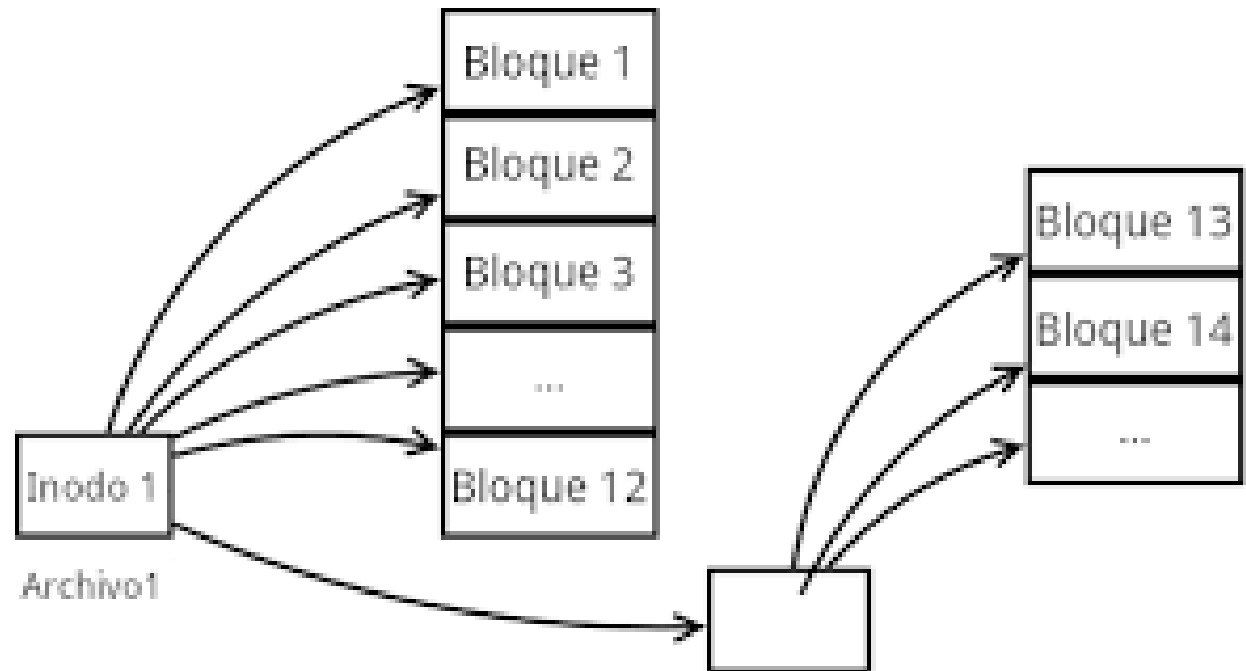
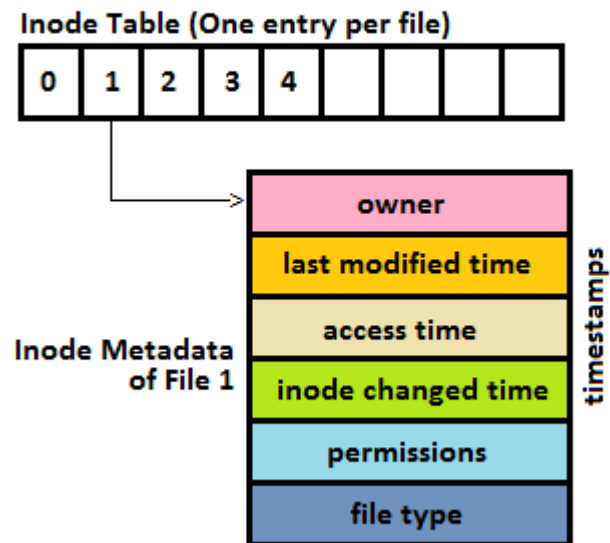
- Borrar enlaces

`unlink enlace`

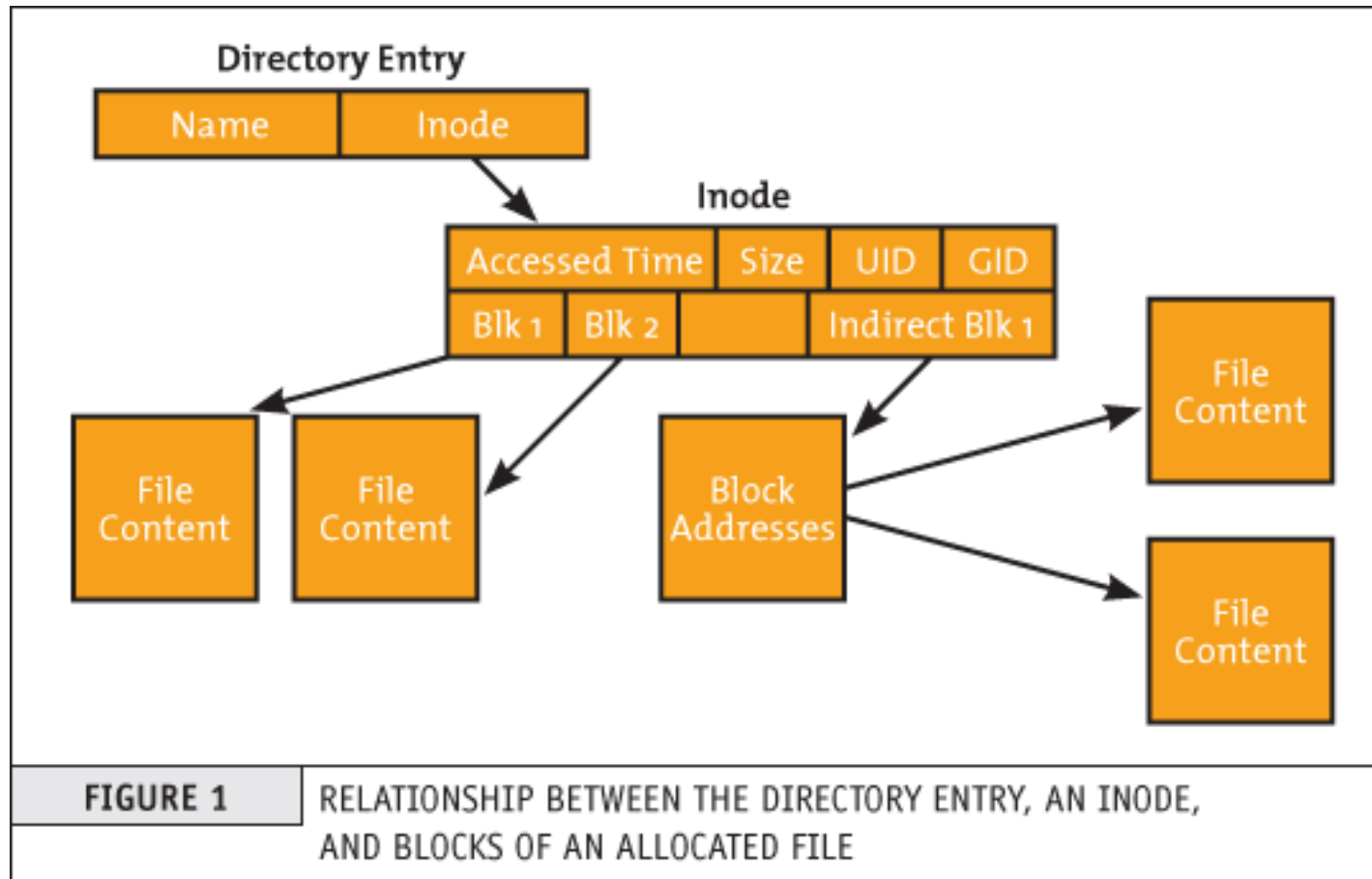
`rm enlace`

Breve explicación de *inodos* en Linux

Inode Entry

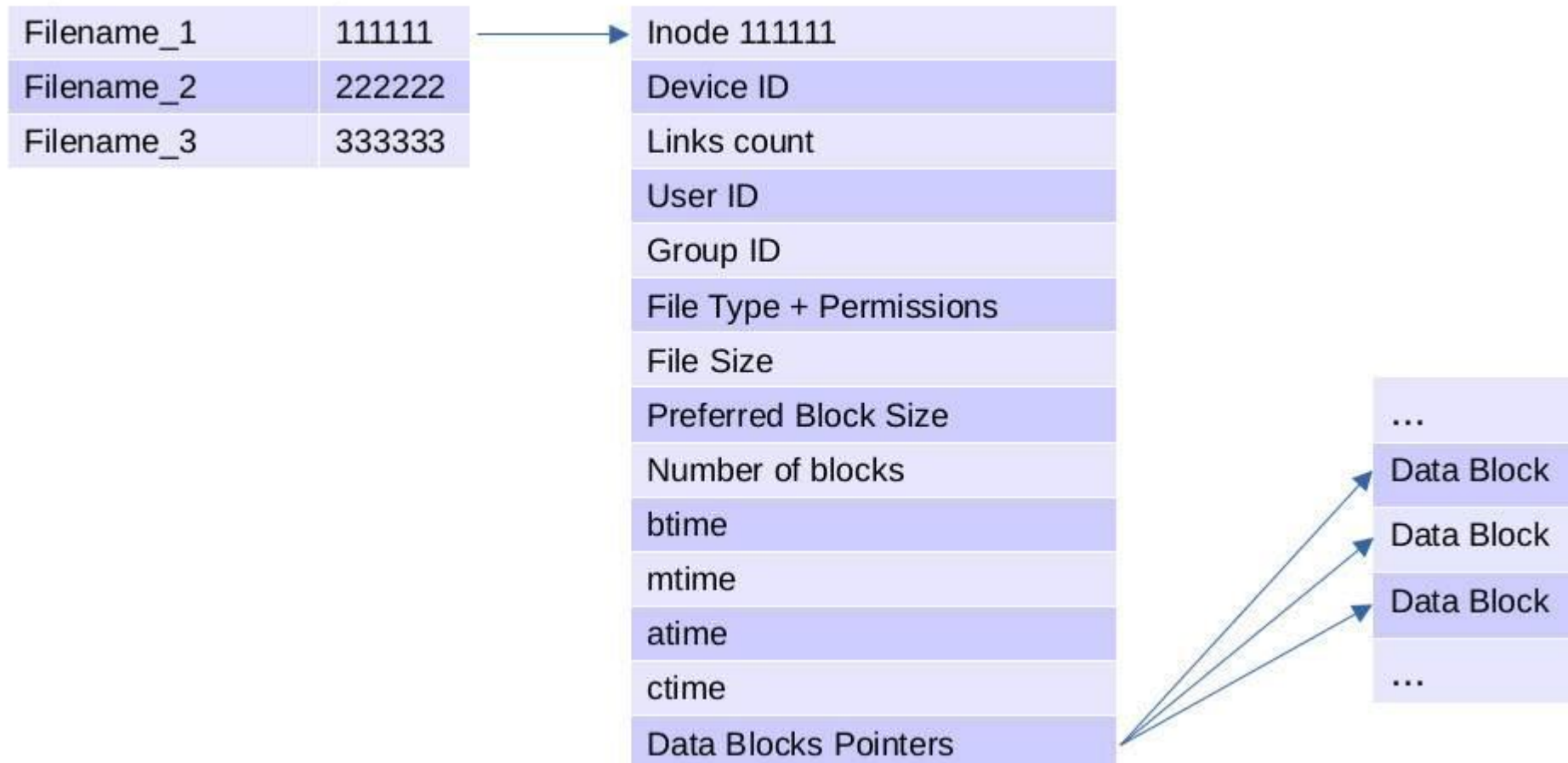


Breve explicación de *inodos* en Linux

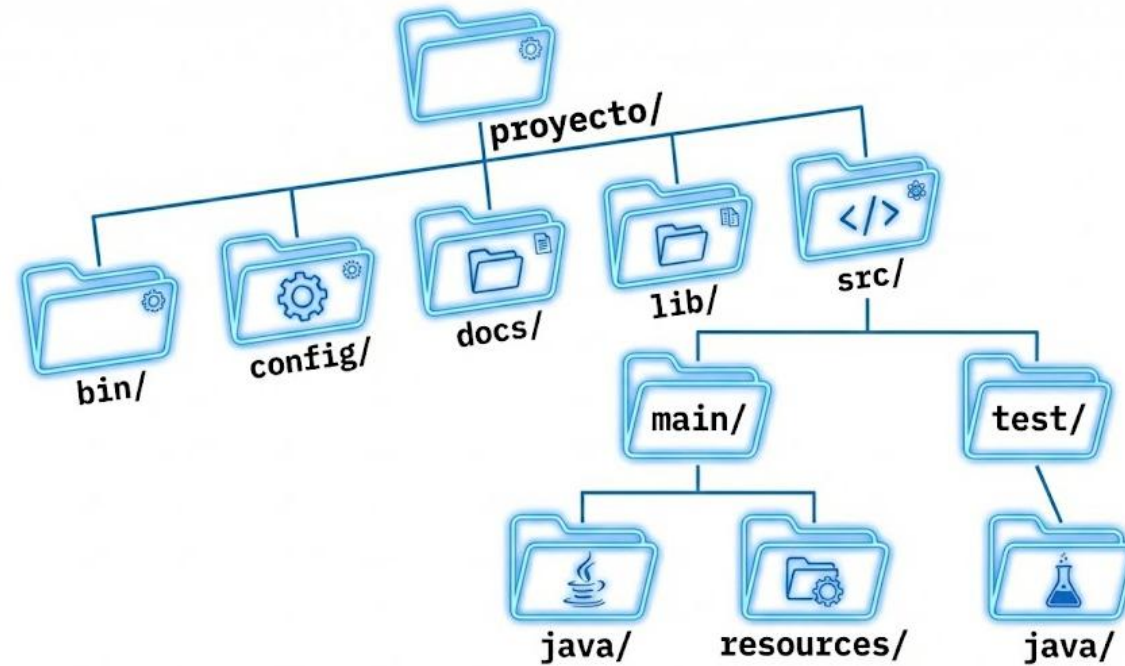


Breve explicación de *inodos* en Linux

/path/to/directory



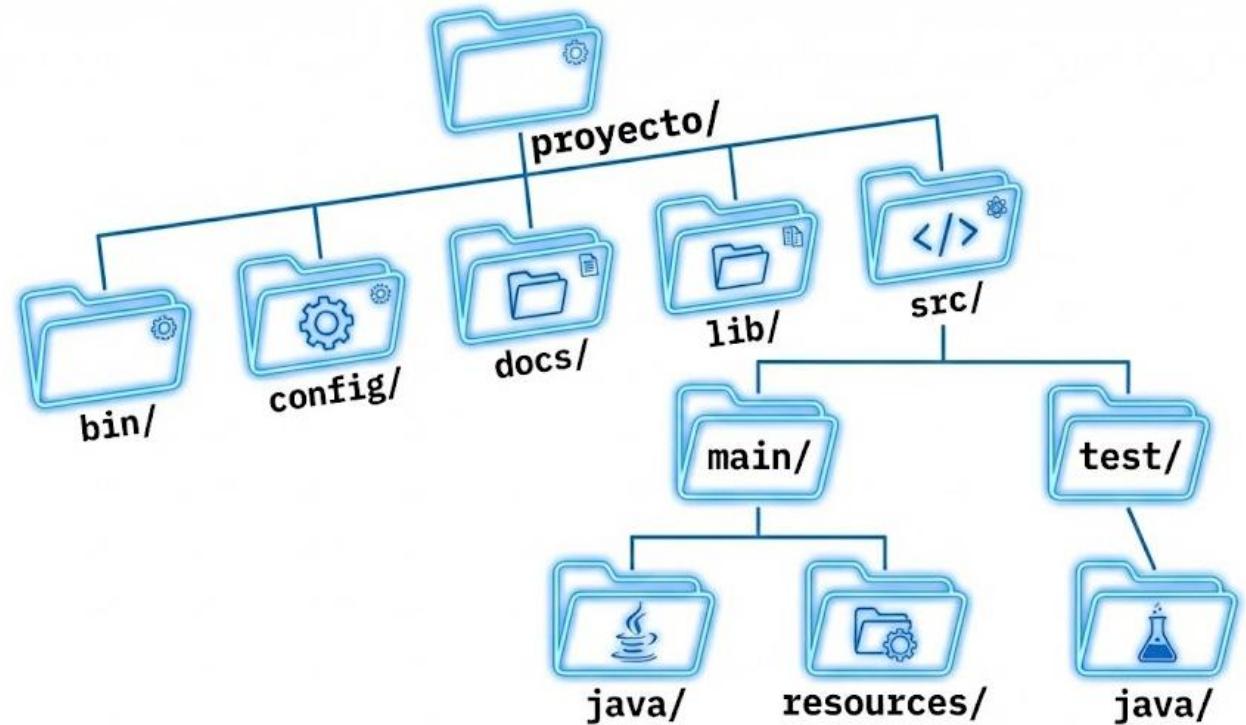
{ Bracing .. Linux }



{ Bracing .. Linux }



¿Serías capaz de crear la estructura de directorios mostrada en la imagen con 1 solo comando en 1 línea?



Bracing Linux

Es una de las funciones más potentes de la **Brace Expansion** (expansión de llaves) en Bash. Los dos puntos (..) se utilizan para generar **secuencias** o rangos. Es una expansión de texto que genera combinaciones antes de ejecutar el comando. No es un “patrón” tipo regex, es el *shell* creando strings.

1. La sintaxis normal: **{valor1,valor2,valor3...}**
2. La sintaxis de rangos: **{inicio..fin}**

`{1..5} → 1 2 3 4 5`

`{01..05} → 01 02 03 04 05`



Bracing Linux

3. La sintaxis **pro** si se requiere con saltos:
{inicio..fin..salto}

{0..10..2} → 0 2 4 6 8 10

{a..d} → a b c d

{Z..N..3} → Z W T Q N



Bracing Linux

4. Encadement o multiplicación, básicamente es iterar y manda la primera línea de llaves empezando por la izquierda.

AULA{1..5}-PUESTO{A..D} → ?



Bracing Linux

SOLUCIÓN:

```
mkdir -p proyecto/{src/{main/{java,resources},test/java},bin,lib,docs,config}
```



**Bash
Scripting**



**¿QUÉ ES UN
SCRIPT?**

Bash Scripting

¿QUÉ ES UN SCRIPT?

Es un programa escrito en texto plano en un archivo que **es interpretado**, no compilado, y que automatiza tareas mediante la ejecución secuencial de instrucciones.

Bash Scripting

¿QUÉ ES UN SCRIPT?

Es un **archivo de texto** que contiene una serie de **instrucciones/comandos** y que representa un pequeño programa. Éste normalmente estará orientado a automatizar ciertas tareas. Se ejecutará por un **intérprete de comandos**: *shell, bash, python, Powershell, JavaScript, Ruby, Node.js, Perl, Lua, etc. (runtime)*

Bash Scripting

¿QUÉ ES UN SCRIPT?

No se suele compilar o por lo general no se compila a un binario antes de ejecutarse, por lo que recalcamos que suele ser interpretado. Además suele ser secuencial también; encadena operaciones y usa herramientas o librerías del entorno donde se ejecuta. Se usa mucho para administración, despliegues, así como para procesamiento de datos y tareas repetitivas y obviamente lo que se te le ocurra al programador. Suelen representar pequeñas herramientas o utilidades.

Bash Scripting

¡Un script vive
del
ecosistema!

No reinventa la rueda, usa
herramientas/comandos
muy potentes del sistema:

- `grep`
- `awk`
- `sed`
- `find`
- `systemctl`
- `find`
- `cd`
- `etc.`

Bash Scripting

¿Por qué deberíamos saber
y aprender scripting?



Bash Scripting | Por qué ?

1 Porque ahorra tiempo real

Si tienes que renombrar 500 archivos manualmente con un patrón, ¿qué haces?

**Un script puede hacerlo en 1 segundo.
Productividad y potencia pura.**

Bash Scripting | Por qué ?

2 Porque convierte al programador en alguien peligroso ;)

Un usuario ejecuta comandos. Un buen informático automatiza el sistema.

Desplegar servidores, crear usuarios, generar backups, analizar logs, lanzar builds, limpiar bases de datos, etc.

¡Esto es poder de ejecución!

Bash Scripting | Por qué ?

3 Porque es la base del DevOp moderno

Sin scripting no existen:

CI/CD, automatización de despliegues, scripts de Docker, control de pipelines, infraestructura como código, etc.

Aunque luego trabajen en Java, C# o web, SIEMPRE habrá scripting por debajo

Bash Scripting | Por qué ?

4 Porque obliga a pensar como el sistema

El scripting nos enseña:

Cómo funciona el sistema, cómo se encadenan procesos, qué es un exit code, cómo fluye la información por stdout y stderr, cómo se gestionan permisos, cómo se comunica el sistema con el hardware, y un largo etcétera.

Es formación estructural, pensamiento transversal y completo.

Bash Scripting | Por qué ?

5 Porque da mentalidad de ingeniero

Un script es:

“No quiero hacer esto 20 veces. Lo hago una vez y lo automatizo.”

Cuando empiezas a escribir scripts empiezas a dejar de usar el ordenador y empiezas a controlarlo.

Bash Scripting ¿Por qué?



Aclaración de conceptos:

Script → es un código no muy extenso que automatiza acciones

Aplicación → Software más estructurado, extenso y modular con varios fines.

Bash Scripting

```
#!/usr/bin/bash
```

```
echo hola muuundooo!!
```

```
echo
```


Bash Scripting

```
#!/usr/bin/bash
```

```
echo Hola muuundooo!!
```

```
echo
```

```
echo
```

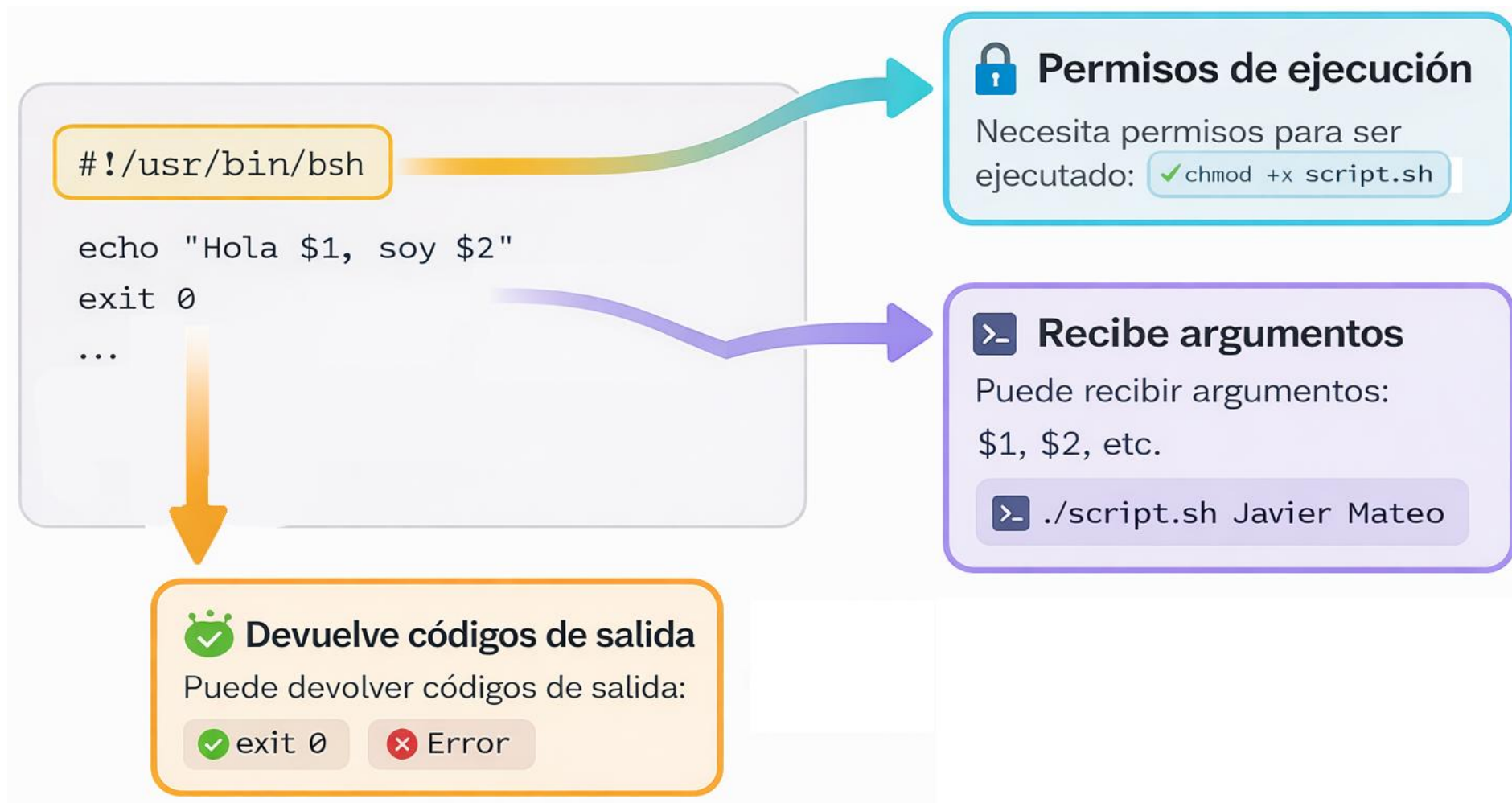
```
...
```

**Primera línea
es el shebang!**

Indica qué intérprete
debe ejecutar el script

**Comandos, código
o instrucciones**

Bash Scripting



Bash Scripting

```
#!/usr/bin/bash
```

```
mi_var="mi mundo crueel"
```

```
echo hola $mi_var
```

```
echo
```

Bash Scripting

```
#!/usr/bin/bash  
  
for f in *.txt; do  
    echo "Procesando $f"  
done
```



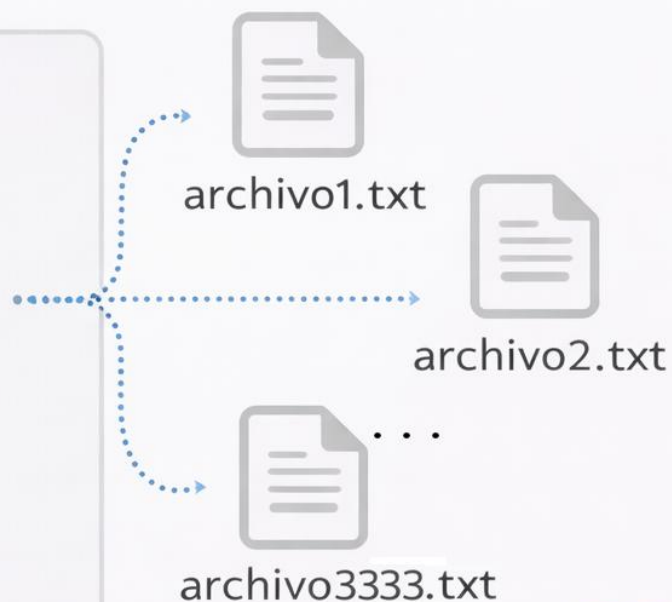
Bash Scripting

```
#!/usr/bin/bash
```

```
for f in *.txt; do
```

```
    echo "Procesando $f"
```

```
done
```



¿Esto lo harías uno por uno?



Bash Scripting

```
#!/usr/bin/bash
```

```
if mkdir -p "$HOME/Descargas/prueba"; then
```

```
    echo "OK, directorio creado (o ya existía)."
```

```
else
```

```
    echo "ERROR, no se pudo crear el directorio." >&2
```

```
fi
```

Bash – Estructuras lógicas

IF

```
if comando_o_test; then
    ...
elif comando_o_test; then
    ...
else
    ...
fi
```

Bash – Estructuras lógicas IF

IF directos comprobando resultado de un comando

```
if mkdir -p /tmp/pepe; then
    echo "Creado (o ya existía)"
else
    echo "Falló mkdir" >&2
fi
```

```
if grep -q "hola" archivo.txt; then
    echo "existe 'hola' en el archivo"
fi
```


Bash – Estructuras lógicas IF

IF directos comprobando resultado de un comando

```
if mkdir -p /tmp/pepe; then
    echo "Creado (o ya existía)"
else
    echo "Falló mkdir" >&2
fi
```

```
if grep -q "hola" fichero.txt; then
    echo "Está 'hola'"
fi
```

Bash – Estructuras lógicas IF

IF con condiciones []

```
if [[ -f "/home/usuario/archivo.txt" ]]; then  
    echo "Es un fichero y existe"  
fi
```

Bash – Estructuras lógicas IF

OPERADORES IF con condiciones []

- **Ficheros:**

- e → ruta existe
- f → ruta existe y es fichero
- d → ruta existe y es directorio
- L → ruta es symlink

- **Strings:**

- z “\$s” → vacío
- n “\$s” → no vacío

- **Números enteros:**

- eq → igual
- ne → no igual
- lt → lower than
- le → lower equal
- gt → greather tan
- ge → greather equal

Bash – Estructuras lógicas IF

IF con condiciones []

```
if [ "$edad" -ge 18 ]; then  
    echo "Es mayor de edad"  
fi
```

```
if [[ "$nombre" == Dav* ]]; then  
    echo "Empieza por Dav"  
fi
```

Bash – Estructuras lógicas IF

IF tipo C/Java

```
if (( var > variable2 )); then  
    echo "La variable $var es mayor que  
$variable2"
```

```
elif (( variable2 == (var+2) )); then  
    echo "mensaje"  
fi
```

Bash – Estructuras lógicas CASE

```
case "$opt" in
    1) echo "Opción 1" ;;
    2) echo "Opción 2" ;;
    3) echo "Opción 3" ;;
    *) echo "Opción inválida" ;;
esac
```

```
# 2) Varias opciones en el mismo bloque (OR)
cmd="$1"
case "$cmd" in
    -h|--help) echo "Ayuda..." ;;
    -v|--version) echo "Versión 1.0" ;;
    *) echo "Opción no soportada" ;;
esac
```

```
# 3) Patrones (globs) tipo extensión de archivo
f="$1"
case "$f" in
    *.jpg|*.png) echo "Es una imagen" ;;
    *.txt)       echo "Es un texto" ;;
    *)           echo "Tipo desconocido" ;;
esac
```

```
s="$1"
case "$s" in
    "" )          echo "Vacío" ;;
    [0-9]* )      echo "Empieza por dígito" ;;
    [[[:alpha:]]* ) echo "Empieza por letra" ;;
    * )           echo "Otro" ;;
esac
```

Bash – Estructuras lógicas CASE

```
# 6) case con "fall-through" (Bash 4+): ;& y ;;&
# ;& ejecuta también el siguiente bloque (sin reevaluar patrón)
# ;;& sigue probando patrones (reevalúa)
x="$1"
case "$x" in
    a)
        echo "es a"
        ;& # también ejecuta el siguiente bloque sí o sí
    b)
        echo "bloque b (o cayó desde a)"
        ;;
    c)
        echo "es c"
        ;;& # sigue probando el resto de patrones
    c*)
        echo "empieza por c"
        ;;
esac
```

Bash – Estructuras BUCLE FOR

```
for x in a b c; do  
    echo "$x"  
done
```

```
arr=("a" "b c" "d")  
for x in "${arr[@]}"; do  
    echo "$x"  
done
```

```
for i in {1..5}; do  
    echo "$i"  
done
```

```
for i in {0..10..2}; do  
    echo "$i"  
done
```


Bash – Estructuras BUCLE FOR

```
# For estilo C/Java (aritméticos)
for ((i=0; i<5; i++)); do
    echo "i=$i"
done

echo

for ((i=10; i>=0; i-=2)); do
    echo "i=$i"
done
```

```
# Dos variables
for ((i=0, j=10; i<5; i++, j-=2)); do
    echo "i=$i j=$j"
done

echo

# Infinito (rompes con break)
for (;;); do
    echo "loop"
    break
done
```

Bash – Estructuras BUCLE WHILE

```
# Leer fichero línea a línea (mejor con while + read)  
while IFS= read -r line; do  
    echo "line=$line"  
done < fichero.txt
```

echo

```
# Leer salida de un comando (process substitution)  
while IFS= read -r line; do  
    echo "line=$line"  
done < <(ls -l)
```

Bash – Control y lectura de datos

LEYENDO DATOS O PARÁMETROS ☺

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
read -r -p "Dime tu nombre: " nombre
```

```
echo "Hola, $nombre"
```

Bash – Leyendo variables con *read*

LEYENDO DATOS O PARÁMETROS 😊

PARÁMETROS DE READ	
-r	Es recomendable y seguro usarlo para que read no escape caracteres si se escribe una \
-s	Modo silent. Silencioso oculta lo que escribes. Se suele usar en contraseñas
-t SECONDS	Hay un tiempo límite de escritura
-p TEXTO	Te permite mostrar un TEXTO directamente a la vez que solicitas el dato. Te ahorras un printf o un echo.
-n N	Solo lee N caracteres. Útil para opciones Sí/No, menús etc.
-a ARRAY	Divide la entrada usando el IFS del sistema y lo guarda en ARRAY
-e	Activa edición con flechas
-i “VALOR INICIAL”	Se usa sobre todo con -e para poder incluir ya un valor por defecto.

Bash – Parámetros de entrada

PARÁMETROS DE ENTRADA

```
#!/usr/bin/bash
```

```
echo "Me pasaste $# parámetros"
```

```
echo "El primero es: $1"
```

```
echo "El segundo es: $2"
```

```
echo "El último fue: ${!#}"
```

```
echo "Todos juntos: $@"
```

Bash – Parámetros de entrada

`./myscript.sh Param1 "Param2 yeah" tercero cuarto etc. Fin`

<code>\$0</code>	El mismo método que ejecutó el script → <code>./myscript.sh</code>
<code>\$1</code>	Devuelve el parámetro en la posición 1 → <code>Param1</code>
<code>\$2</code>	Devuelve el parámetro en la posición 2 → <code>Param2 yeah</code>
<code>\$5</code>	Devuelve el quinto parámetro → <code>etc.</code>
<code>\$@</code>	Todos los parámetros de entradas separados en cadenas individuales tal cual los introdujo el usuario. Es lo que se us para recorrerlos con un bucle. → <code>Param1 Param2 yeah tercero cuarto etc. Fin</code>
<code>\$#</code>	Número de parámetros totales recibidos. → <code>6</code>
<code>\${!#}</code>	Te devuelve el último parámetro recibido → <code>Fin</code>
<code>\$*</code>	Todos los parámetros juntos en formato <i>string</i> sin deferencias de como se pasaron. → <code>Param1 Param2 yeah tercero cuarto etc. Fin</code>

Bash – Control de Errores

Longitud de variables `${#variable}`

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
texto="Hola Mundo"
```

```
echo ${#texto}
```

```
Resultado: 10
```

Bash – Control de Errores

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
cd ruta/ruta
```

```
rm *
```

```
exit 0
```


Bash Scripting - Funciones

```
mi_func() {  
    local x="solo aquí"  
    echo "dentro: $x"  
}
```

```
mi_func  
echo "fuera: $x"
```

Bash Scripting - Funciones

```
mi_func() {  
    echo "recibo parámetros como el script"  
    echo "$1"  
    echo "$2"  
    echo "Aquí dentro no conozco los parámetros del  
script salvo que los hayas guardado"  
}
```

```
mi_func param1 param2
```

Bash Scripting - Funciones

```
sumar() {  
    echo $(( $1 + $2 ))  
}
```

...

```
sumar 5 7      # Esto dará el resultado y lo imprimirá
```

...

```
res="$ (sumar 5 7) "
```

Bash Scripting – Expansión de parámetros

```
res = ${variable//PATRON/REEMPLAZO}
```

coge el contenido de variable

reemplaza PATRON por REEMPLAZO

Si antes de PATRON fuera / (una sola) rsolo reemplaza la primera aparición
y devuelve el resultado en **res**

```
# Reemplazar http por https (todas las apariciones)
url="http://miweb.com/pagina?redir=http://otro.com"
nueva=${url//http:\\\\https:\\\\}
echo "$nueva"
# https://miweb.com/pagina?redir=https://otro.com
```

```
# Solo la primera aparición (una sola /)
url="http://miweb.com/pagina?redir=http://otro.com"
nueva=${url/http:\\\\https:\\\\}
echo "$nueva"
# https://miweb.com/pagina?redir=http://otro.com
```

Bash Scripting – Expansión de parámetros

- **Sustitución**

`${var/pat/rep}` (solo primera)

`${var//pat/rep}` (todas)

- **Recorte (trim) por patrón**

Desde el inicio:

`${var#pat}` quita el match más corto al principio

`${var##pat}` quita el match más largo al principio

Desde el final:

`${var%pat}` quita el match más corto al final

`${var%%pat}` quita el match más largo al final

Bash Scripting – Expansión de parámetros

Para un fichero en concreto siendo por ejemplo
`file="/home/usuario/mifichero.jpg"`

`${file##*/}` (basename) → `mifichero.jpg`

`${file%/*}` (dirname simple) → `/home/usuario`

`${file##*.}` (extensión) → `.jpg`

`${file%.*}` (sin extensión) → `/home/usuario/mifichero`

Bash Scripting – The big FIVE

- **GREP**

```
grep "/bin/bash" /etc/passwd
```

Busca líneas que coincidan con un patrón. Filtra el texto por el contenido. Acepta expresiones regulares y muchos parámetros. Ideal para “búsquedas” dentro de ficheros.

- **CUT**

```
cut -d: -f1 /etc/passwd
```

Corta columnas de cada línea por delimitador o por posiciones. Muy rápido para campos simples. Uso típico: “saca la 1ª y 3ª columna separada por :

- **TR**

Traduce o borra caracteres, trabaja carácter a carácter (no entiende “líneas” ni regex). Uso típico: pasar a minúsculas, quitar \r, eliminar caracteres, colapsar sets.

```
# tr: transforma caracteres (aquí, pasa a mayúsculas un texto)  
echo "hola clase" | tr '[:lower:]' '[:upper:]'
```

Bash Scripting – The big FIVE

- **SED**

```
grep "/bin/" /etc/passwd | sed 's/bash/TORTUGA/g'
```

Editor de flujo, transforma texto línea a línea (sustituciones, borrados, inserciones) con regex. Uso típico: “reemplaza X por Y”, “borra líneas que cumplan”.

- **AWK**

```
awk -F: '{print $1 " -> " $7}' /etc/passwd
```

¡Procesado por campos muy potente!, muchos lo consideran casi un mini lenguaje (variables, condiciones, cálculos). Es el más completo para “parsing” de texto estructurado. Uso típico: “suma una columna”, “filtra por columna 3”, “reformatea un CSV/log”.

Aplicaciones

Programas

Procesos

threads

Servicios

Aplicaciones vs Programas

Aplicaciones y Programas

En el uso cotidiano, solemos usar indistintamente programa o aplicación casi como sinónimos. Y aunque ambos términos se refieren al software instalado (almacenado en disco) y preparado para ejecutarse, hay diferencias:

- **APLICACIÓN:** suele usarse más para programas **orientados al usuario final** (muchas veces con interfaz gráfica, aunque no tiene porqué). Subconjunto dentro de “programas” pero orientados al usuario final.
- **PROGRAMA:** es el concepto genérico: conjunto de instrucciones que la CPU ejecuta. Puede ser: *un driver, un pequeño script, o un programa más complejo*. No tiene por qué tener fin ni interacción directa.

Procesos

Procesos

Un proceso es una instancia de un programa que se está ejecutando en el sistema en un momento dado.

Un **programa** se transforma en **proceso** en el momento en que éste se ejecuta y pasa a estar cargado en la memoria del sistema.

En ese momento “el proceso” recibe un PID (número identificativo por parte del SO) y se asocia dicho PID con un nombre que suele coincidir con el nombre del programa que está ejecutando.

¿Qué es un *Proceso*?

El proceso es la unidad que usa el sistema operativo para **gestionar la ejecución de un programa**. No es solo el código del programa, un proceso implica que el programa ha pasado a ejecución por lo que también incluye todo lo que necesita éste para funcionar mientras se está ejecutando:

- **Identificador único o PID**
- Su **memoria**
- Su **estado** (ejecutándose, esperando, detenido, etc.)
- Su **prioridad**
- **Recursos** que está usando: CPU, RAM, archivos, dispositivos...
- Información del **proceso padre**

NOTA: El **SO** almacena todo esto en la **PCB** (Process Control Block)

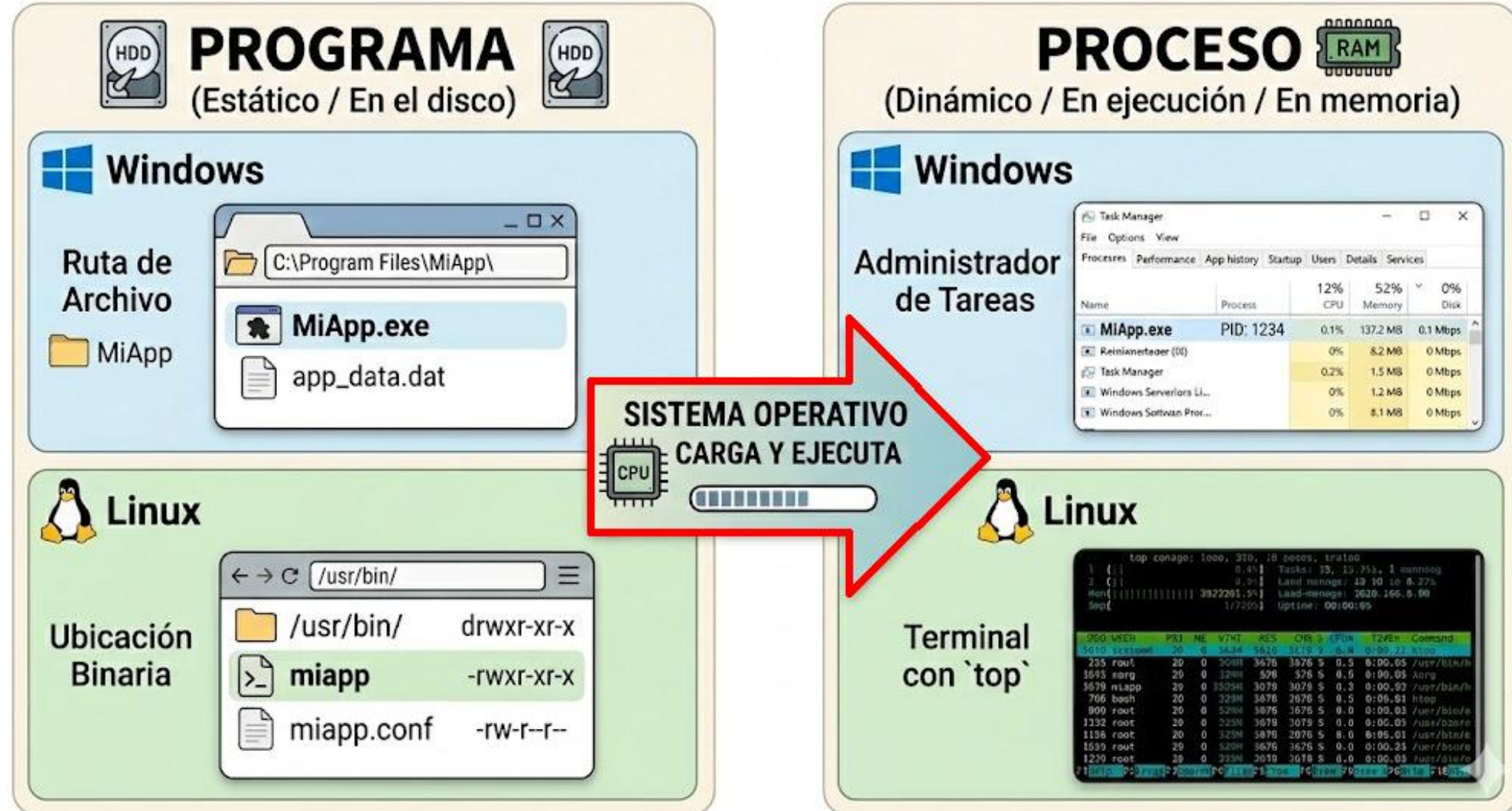
Programa vs Proceso

Programa y Proceso

- **Programa**: Los archivos de dicha aplicación en el disco.
- **Proceso**: Ese mismo programa se convierte en proceso cuando el sistema operativo lo carga y lo pone en ejecución.

El proceso es el programa en ejecución, junto con todos los recursos e información que el sistema operativo necesita para ejecutarlo y controlarlo.

PROGRAMA → PROCESO



Programas y Procesos

MATICES: Una **app** no siempre equivale a un solo proceso

- Una app cerrada sigue siendo un programa instalado, pero **no es un proceso** hasta que se ejecuta.
- Una app abierta puede tener **uno o varios procesos**. *Ejemplos: Chrome, Chromium, Edge, Firefox... suelen lanzar varios procesos.*

Todo proceso proviene de un programa, pero no todo programa es un proceso

Un programa es algo pasivo, almacenado en disco. Un proceso es algo activo, en ejecución y controlado por el sistema operativo.

¿Y los scripts?



¿Y los scripts?

Un script no es un proceso, sino un archivo de texto con instrucciones que interpreta otro programa, como Bash, PowerShell o Python.

Cuando se ejecuta, el intérprete lo lee y lo ejecuta, dando lugar a uno o varios procesos.



Threads o hilos

Threads (hilos) en Procesos

- Un **hilo o thread** es una **unidad de ejecución dentro de un proceso**.
- Un mismo proceso puede tener **uno o varios hilos**, que comparten la **misma memoria y recursos** del proceso, pero pueden ejecutar tareas de forma concurrente.

Un hilo es una unidad de ejecución dentro de un proceso

Ejemplos de comandos

COMANDO	DESCRIPCIÓN
ps	Muestra sólo los procesos de la terminal/sesión actual y del usuario actual.
ps ux	Muestra todos los procesos del usuario actual incluyendo aquellos que no tienen terminal asociada: servicios, procesos en segundo plano, aplicaciones del entorno gráfico.
ps aux	Todos los proceos de Todos los usuarios. No se guarda nada 😊. Formato BSD.
ps aux --sort=...	Tanto ps ux como ps aux nos permite ordenar con --sort= por una columna.
pstree	Muestra una vista tipo árbol de todos los procesos . Útil para identificar orígenes.
pstree -p	Le decimos que nos incluya el PID de cada proceso.
pstree usuario	Sólo los procesos del usuario dado.
pstree PID	Sólo los procesos del proceso padre con PID dado. “Zoom máximo”.
pstree \$\$	\$\$ devuelve el PID de la terminal actual.
pgrep	Muestra procesos que coincidan con el patrón dado. Se puede también pgrep -u



BSD FORMAT: *u → User-Oriented incluye información extra. x → eXtended incluye todos los procesos que no tienen una terminal asociada por ejemplo servicios y daemons del sistema*

Comandos - Ejemplos

Abre el navegador (Firefox, Chrome , Edge) en tu entorno de Escritorio y prueba los diferentes comandos vistos en una terminal y observa las diferencias, así como todo lo aprendido en clase:

ps

ps ux

pstree usuario

pstree -p usuario

- ¿Eres capaz de identificar el PID, ¿ves el uso de CPU y MEMORIA?
- Ves los diferentes procesos de un programa Firefox? ¿Y sus hilos?
- Abre ahora otra terminal y ejecuta el **man** sobre un comando cualquiera, y mira los procesos desde la otra terminal. ¿Qué podemos ver?

Planos de ejecución

Planos de ejecución

Un proceso se puede ejecutar en primer plano o en segundo plano:

- **Primer plano:**

Es el proceso con el que el usuario interactúa directamente. Decimos que el proceso está en foreground (fg).

- **Segundo plano:**

Es un proceso que se ejecuta en la "sombra", sin necesidad de interacción directa ni de ocupar la interfaz principal. Decimos que está en background (bg).

Procesos - Planos de ejecución

- **Primer plano:** Cualquier comando que lancemos de forma normal en una terminal se ejecutará en *foreground*.
- **Segundo plano:** Añadimos el **&** al final para decirle que lo ejecute en background.

```
</> Bash
```

```
comando &
```

```
</> Bash
```

```
firefox &
```

Procesos - Planos de ejecución

- **De FG a PAUSA** → **CTRL+Z** → Si un proceso en primer plano tarda mucho, pulsas Ctrl + Z. El proceso se detiene (queda "congelado" en RAM)
- **De PAUSA a BG** → Escribes el comando **bg** (background). El proceso que acabas de pausar se "descongela" y sigue trabajando, pero ya no te bloquea la terminal.
- **De BG a FG** → Si quieres volver a interactuar con él, usas el comando **fg** (foreground) y vuelve a "secuestrar" tu pantalla.

Procesos - Planos de ejecución

USOS	LO QUE OCURRE
comando &	Lanza el proceso del comando directamente en background.
CTRL+C	Manda a PARAR/FINALIZAR el proceso que está en primer plano (foreground). SIGINT.
CTRL+Z	SUSPENDE / PAUSA el proceso actual (lo deja Stopped). SIGSTOP.
jobs jobs -l	Muestra el listado de procesos gestionados por la terminal actual
bg bg ID	Reanuda un proceso pausado, pero en segundo plano.
fg fg ID	Trae un proceso del <i>background</i> al <i>foreground</i> , primer plano.
nohup	Permite que un proceso siga ejecutándose, aunque se cierre la terminal.

Procesos - Comando *kill*

El comando **kill** lo que hace es enviar una SEÑAL a un proceso usando su PID.

SEÑALES KILL	¿QUÉ HACE?
SIGTERM -15	Señal por defecto si no pones nada. Le pide al proceso que termine de forma “bien”, ordenada y el proceso debe intentar cerrar y terminar de forma correcta. Puede ignorarse.
SIGKILL -9	Mata forzosamente el proceso sin preguntar. No le da tiempo a respirar. Cuidado: mejor usarlo si el primero no funciona y el proceso está colgado y con cuidado porque se pueden perder datos.
SIGINT -2	Interrumpe el proceso. La típica que mandamos con un CTRL+C en terminal. Interrumpe el proceso y el proceso la puede ignorar.
SIGSTOP -19	Se detiene (se congela) el proceso, se para pero no termina, digamos que se suspende. Aunque el proceso debe manejarla.
SIGCONT -18	Reanuda un proceso detenido con SIGSTOP. Continuaría por donde se había quedado congelado si el programa lo contempla.
SIGHUP -1	Recargar configuración aunque depende de cada proceso.

Procesos – kill



Procesos – kill - Ejemplos

- Ejecuta: **nano loquesea.sh**
- Escribe algo dentro sin guardar
- Prueba desde otra terminal a hacerle un **kill** normal (SIGTERM) al proceso. Debes buscar su PID con `ps ux` o `pstree -p`, lo que prefieras.
- Repite el proceso con otro nombre: **nano loquesea2.sh**
- ¿Y ahora qué pasa si hacemos un **kill -9**?



Procesos – comando *kill*

- Otros comandos kill:

Alternativas a KILL	¿Qué hace?
pkill	“Mata” por nombre o patrón en vez de por PID. Con -f “que contenga en su ruta”
pkill firefox pkill -9 firefox pkill -f firefox pkill -u David	
killall	“Mata” por nombre pero todos los procesos que el nombre coincida.
killall firefox	
xkill	“Mata” una ventana con el ratón

APLICACIÓN, PROGRAMA → PROCESO → HILO (THREAD)

APLICACIÓN



Software instalado
o almacenado **en disco**

PROGRAMA



Archivo o software
almacenado en disco

PROCESO



Programas en ejecución
(**PID, memoria, estado**)
→ Uno o varios procesos

HILO (THREAD)



Unidades de ejecución
que comparten **memoria**

Estados de un Proceso

¿Qué son los estados de un proceso?

Un proceso no “existe” en plan mágico haciendo cosas todo el rato. El **SO** tiene cientos a la vez, y necesita **clasificarlos** para poder decidir:

- **Quién usa CPU ahora** (planificación, scheduler).
- **Saber por qué uno no avanza** (está esperando disco, red, teclado...).
- **Gestionar recursos** (memoria, I/O, permisos).
- **Depurar** (ver cuelgues, bloqueos, “va lento”, “no responde”).
- etc.

Estados de un proceso

De forma general:

Nuevo → Listo → Ejecutando → (Listo / Esperando / Parado) →
Terminado

Estados de un **Proceso** en **Windows**

**R****Running**

- ➔ Ejecutándose activamente
- ➔ Usando la CPU

W**Waiting**

- ➔ Esperando un evento
- ➔ En espera de recursos

S**Sleeping**

- ➔ Suspendido o en reposo
- ➔ En estado de espera

I**Idle**

- ➔ Inactivo, sin tareas
- ➔ Esperando actividad

T**Terminated**

- ➔ Proceso finalizado
- ➔ Terminó y se cerró

Z**Zombie**

- ➔ Proceso muerto, residuo
- ➔ Cerrado pero sigue en memoria

Estados de un **Proceso** en **Linux**

**R****Running /
Runnable**

- En CPU o listo para ejecutarse



→ En CPU o listo para ejecutarse

S**Interruptible Sleep**

→ Espera entrada o respuesta

D**Uninterruptible Sleep**

- Bloqueado, espera crítica E/S

**I****Idle**

→ Inactivo, sin trabajo pendiente

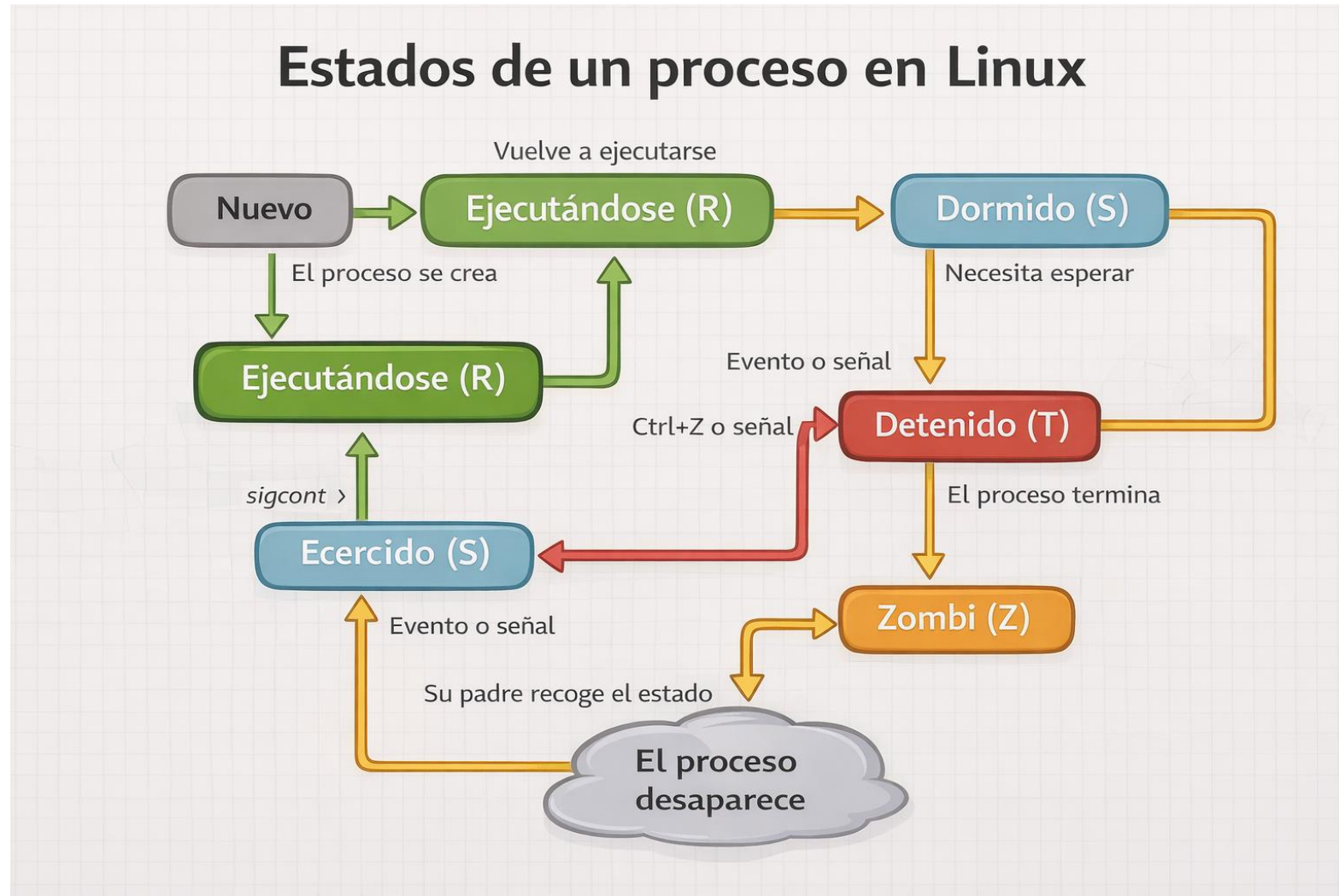
T**Stopped**

→ Detenido o pausado

Z**Zombie**

→ Terminó pero no finalizó

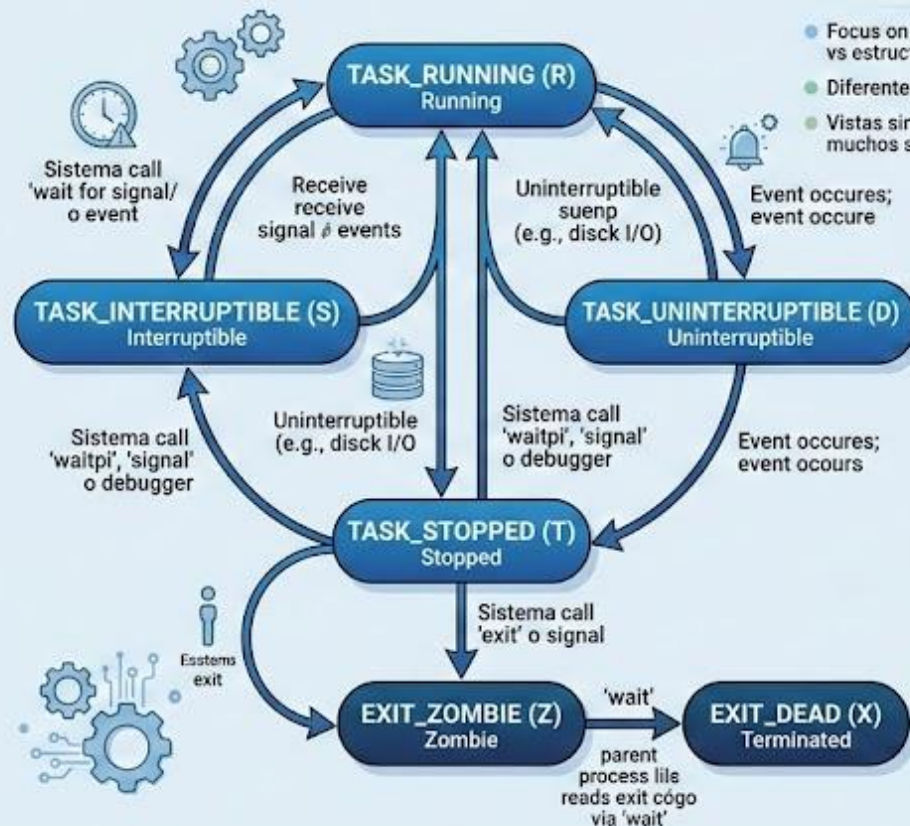
Procesos – Estados GNU/Linux



Procesos – Estados Windows

COMPARACIÓN DE ESTADOS DE PROCESO: LINUX VS WINDOWS

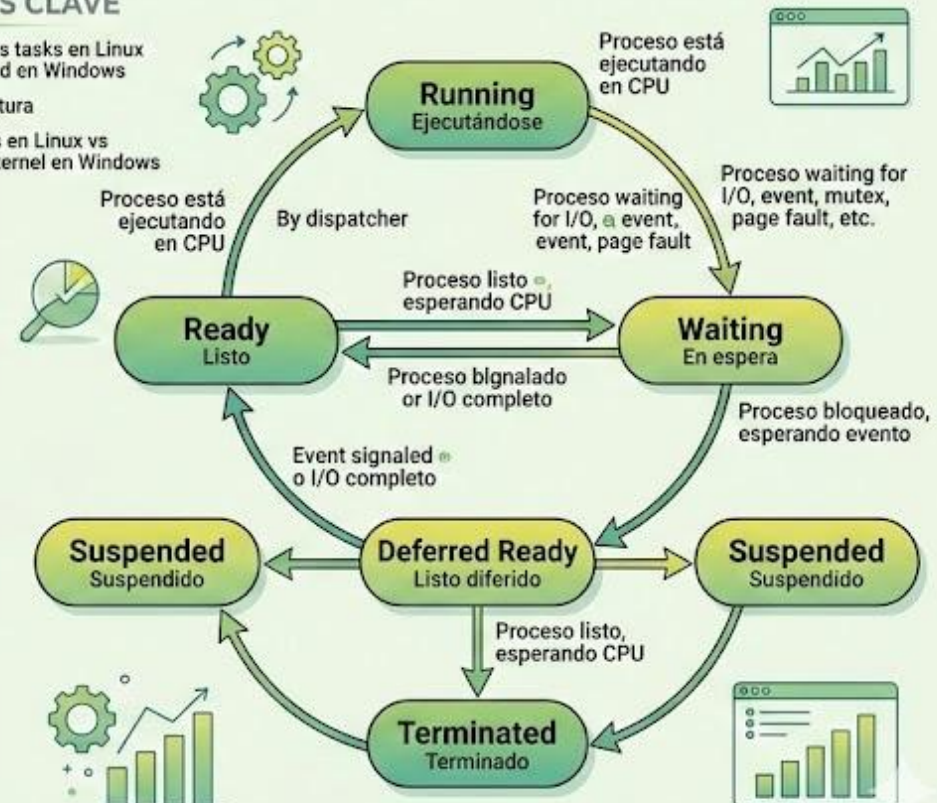
ESTADOS DE PROCESO EN LINUX



DIFERENCIAS CLAVE

- Focus on estructuras tasks en Linux vs estructuras thread en Windows
- Diferente nomenclatura
- Vistas simplificadas en Linux vs muchos estados de kernel en Windows

ESTADOS DE PROCESO EN WINDOWS



Procesos – Estados del Proceso Linux

El comando `ps` nos muestra el **estado** de cada proceso mirando la columna **STAT** siendo la primera letra la más importante:

STAT	DESCRIPCIÓN DEL ESTADO
R	Running/Runnable: el proceso se está ejecutando en la CPU o está listo para ejecutarse en cuanto le toque turno.
S	Interruptible Sleep: el proceso está en espera de algún evento, como entrada del teclado, llegada de datos o una respuesta del sistema. Es el estado más habitual.
D	Uninterruptible Sleep: el proceso está bloqueado esperando una operación crítica, normalmente de entrada/salida, como acceso a disco. No responde a señales normales mientras está en ese estado.
I	Idle: hilo del kernel inactivo, sin trabajo que realizar en ese instante.
T	Stopped: el proceso ha sido detenido, normalmente por control de tareas o por una señal, por ejemplo, al pulsar CTRL+Z.
Z	Zombie: el proceso ya ha terminado, pero sigue apareciendo porque su proceso padre todavía no ha recogido su estado de finalización. El padre no ha recogido el cadáver.

Procesos – Estados del Proceso Linux

El comando *ps* nos muestra el **estado** de cada proceso mirando la columna **STAT**. Pero además con otras letras y símbolos importantes los **ATRIBUTOS**:

extra	DESCRIPCIÓN
<	Proceso con alta prioridad , al que el sistema favorece frente a otros.
N	Proceso con baja prioridad , ejecutado de forma más “amable” con el resto.
s	líder de sesión , normalmente el proceso principal de una sesión o terminal.
l	proceso multihilo , es decir, con varios hilos de ejecución.
+	proceso en primer plano , asociado a la terminal activa.
Z	el proceso ya ha terminado, pero sigue apareciendo porque su proceso padre todavía no ha recogido su estado de finalización. Ojo un proceso zombie no se puede matar ni con kill -9, sólo se conseguiría matando al padre o esperando que éste lo “recoja”.



La columna `STAT` no solo dice **qué está haciendo** un proceso, sino también **cómo se está ejecutando** o qué papel tiene dentro del sistema.

Procesos – Estados del Proceso Linux

Más ejemplos de lo que podríamos ver:

EJEMPLOS	
Ss	Dormido + líder de sesión.
R+	Ejecutándose + primer plano.
Ssl	Dormido + líder de sesión + multihilo.
I<	Proceso o hilo del kernel inactivo con alta prioridad.
RI	Ejecutándose y es un proceso multihilo
T+	Detenido y en primer plano.
SN	Dormido y con baja prioridad.
Zs	Zombie que además era líder de sesión.

Prioriad de procesos

Procesos – nice y renice

Los comandos **nice** y **renice** son los comandos de Linux que controlan la prioridad que tiene cada proceso sobre el resto al ejecutarse y por lo tanto influye directamente en la prioridad del planificador de procesos.

¿Qué es el “nice” de un proceso?

Es un **valor numérico**, que va desde **-20 a 19 (por defecto es 0)** y el cual nos indica lo “amable” que es un proceso. Realmente influye en la prioridad de planificación que le asociará la CPU a dicho proceso.

Cuanto **más alto** es el valor del **nice**, MENOS PRIORIDAD tiene.

Cuanto **más bajo** es el valor del **nice**, MÁS PRIORIDAD tiene.

Procesos – nice y renice

Cuando un proceso es “más nice” decimos que es más educado, amable, deja pasar a otros antes y por lo tanto tiene menos prioridad.

Cuando un proceso es “menos nice”, es más agresivo, se pide más recursos que los demás y tiene más prioridad que el resto de procesos del sistema.

Procesos – nice y renice



Procesos – nice y renice

Ejemplos:

```
nice -n 10 comando # Arranca con nice=10, menos prioridad de CPU
nice -n 15 tar czf copia.tar.gz /home/david # Compresión "suave", molesta menos al sistema
nice -10 comando # Variante menos clara para nice negativo, mejor -n -10
nice -n -10 comando # Arranca con nice=-10, más prioridad (suele requerir sudo)
renice 5 -p 1234 # Cambia PID 1234 a nice=5, le baja prioridad
sudo renice -5 -p 1234 # Cambia PID 1234 a nice=-5, le sube prioridad (sudo)
ps -o pid,ni,%cpu,%mem,comm -p 1234 # Ver NI y consumo de CPU/MEM del proceso
```

SERVICIOS

Servicios

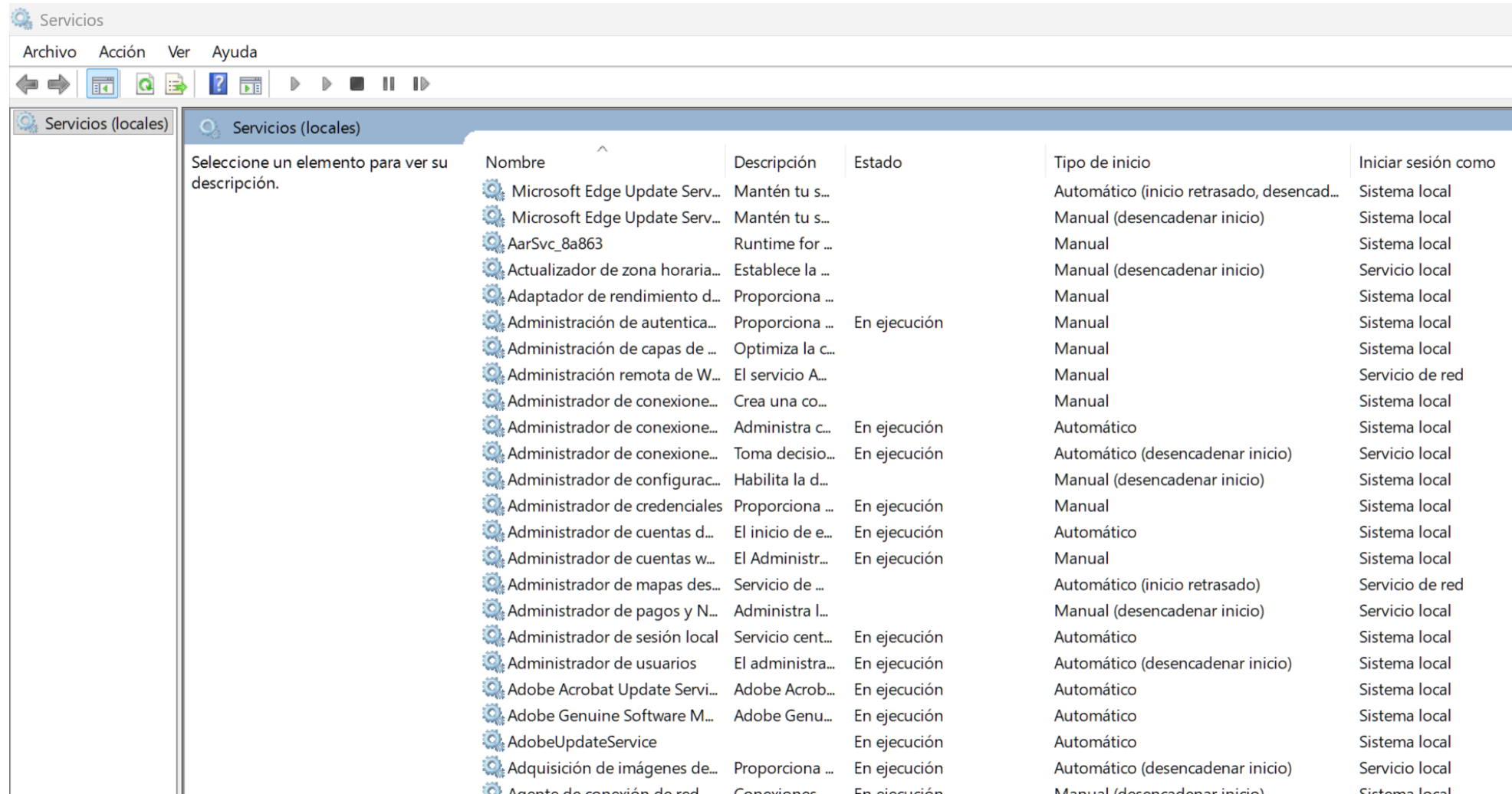
Un servicio es un programa que el sistema operativo ejecuta y gestiona, normalmente en segundo plano, para ofrecer una función al sistema, a otros programas o a la red.

Servicios

- Suelen ejecutarse en **background**
- Normalmente **no interactúa directamente con el usuario**
- Puede y suele arrancar **automáticamente** con el sistema
- El sistema puede **iniciarlo, detenerlo, reiniciarlo** y por supuesto, **consultar su estado**

Servicios en Windows

Services.msc



Servicios en Windows

EJEMPLOS DE SERVICIOS DE WINDOWS

- **Windows Update:** gestiona actualizaciones.
- **Print Spooler:** maneja la cola de impresión.
- **DHCP Client:** el encargado de la configuración de red automática.
- **DNS Client:** resolución de nombres de red.
- **Windows Audio:** servicio de sonido.
- **Task Scheduler:** programación de tareas y programas.
- **Remote Desktop Services:** El encargado del escritorio remoto.
- **Windows Defender:** Seguridad y antivirus.
- **Bluetooth Support Service:** Soporte Bluetooth.
- **Event Log:** Registros de eventos del sistema.

Servicios en GNU/Linux

systemctl list-units --type=service

```
david@debian-server:~$ systemctl --type=service
```

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
apparmor.service	loaded	active	exited	Load AppArmor profiles
console-setup.service	loaded	active	exited	Set console font and keymap
containerd.service	loaded	active	running	containerd container runtime
cron.service	loaded	active	running	Regular background program processing daemon
dbus.service	loaded	active	running	D-Bus System Message Bus
docker.service	loaded	active	running	Docker Application Container Engine
getty@tty1.service	loaded	active	running	Getty on tty1
ifup@ens18.service	loaded	active	exited	ifup for ens18
ifupdown-pre.service	loaded	active	exited	Helper to synchronize boot up for ifupdown
keyboard-setup.service	loaded	active	exited	Set the console keyboard layout
kmod-static-nodes.service	loaded	active	exited	Create List of Static Device Nodes
networking.service	loaded	active	exited	Raise network interfaces
nfs-blkmap.service	loaded	active	running	pNFS block layout mapping daemon
qemu-guest-agent.service	loaded	active	running	QEMU Guest Agent
rpc-statd-notify.service	loaded	active	exited	Notify NFS peers of a restart
rpcbind.service	loaded	active	running	RPC bind portmap service
serial-getty@ttyS0.service	loaded	active	running	Serial Getty on ttyS0
ssh.service	loaded	active	running	OpenBSD Secure Shell server
systemd-binfmt.service	loaded	active	exited	Set Up Additional Binary Formats
systemd-journal-flush.service	loaded	active	exited	Flush Journal to Persistent Storage
systemd-journald.service	loaded	active	running	Journal Service
systemd-logind.service	loaded	active	running	User Login Management
systemd-modules-load.service	loaded	active	exited	Load Kernel Modules
systemd-random-seed.service	loaded	active	exited	Load/Save OS Random Seed
systemd-remount-fs.service	loaded	active	exited	Remount Root and Kernel File Systems
systemd-sysctl.service	loaded	active	exited	Apply Kernel Variables
systemd-timesyncd.service	loaded	active	running	Network Time Synchronization
systemd-tmpfiles-setup-dev-early.service	loaded	active	exited	Create Static Device Nodes in /dev gracefully

lines 1-29... skipping...

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
apparmor.service	loaded	active	exited	Load AppArmor profiles

Servicios en Linux

EJEMPLOS DE SERVICIOS DE LINUX

Gestionados en sistemas modernos por systemd y controlados por systemctl

- **sshd**: acceso remoto por SSH
- **cron**: ejecución programada de tareas
- **nginx**: servidor web
- **NetworkManager**: gestión de red.
- **docker**: gestión de contenedores Docker.
- **cups**: servicio de impresión de Linux, etc.

Con interfaz gráfica hoy en día tenemos apps como stacer y systemdGenie de KDE, ambas nacens sobre el 2017

Servicios vs Proceso

- **Proceso:** cualquier programa en ejecución.
- **Servicio:** programa gestionado por el sistema para ofrecer una función continua, periódica o de soporte. En un servicio se define: cómo se arranca, cuando, cómo se para, si se reinicia si falla, orden de arranque, de qué depende, qué usuario lo ejecuta, qué binario lanza (daemon), etc. Archivos .service

Todo servicio en ejecución tiene uno o varios procesos, pero no todo proceso es un servicio.

No todo proceso en segundo plano es un servicio.

SERVICIOS

SERVICIOS

VS

DAEMONS

DAEMONS

COMPRENSIÓN DEL
FUNCIONAMIENTO INTERNO
INTERNO DE LINUX

```
systemctl --dpasinate daemons  
# daemons
```

```
$ ps -ef  
systemctl --syncsystems daemons  
systemctl * eo  
$ sync systemctl +a.com/piontimendor; -eit  
$ systemctl --not daemons  
$ systemctl --activity daemons
```

```
systemctl proces daemons...  
systemctl match/foremont.comntity_target ()  
systemctl maintlsstior()
```

```
3.41M037:8tlo: systemctl()
```

```
Nov 29 20:28:26 logic /psc/sar/proceess: systemctl>eleate.io --deph  
Nov 29 20:28:29 daemons [down/s/a/28ea3at2: systemctl( systemctl *('daemon'  
Nov 29 20:28:25 logic /server/haveans/daemons daemons! 10.72 linection 0N33:19  
Apr 29 20:28:35 login los  
Mar 29 20:28:35 login loc  
Apr 29 20:28:35 login los
```

```
ps -ef
```

```
systemctl/daemons/daemons  
-s /systemctl/daemons/daemons)  
-h /systemctl/daemons/daemons)  
-B /systemctl/daemons/daemons)
```


¡Un Daemon NO es un Demonio!

En Linux, un "Daemon" es algo bueno que te ayuda.

¡Siempre
te Asisto!

¡Todo
en Orden!

Linux **DAEMON** ✓

¡Soy
Malvado!

¡Nada que ver
commigo!

DEMON maligno ✗



¡Hola!, estoy aquí
para ayudarte!

Linux daemons

En **Unix y GNU/Linux** se usa para procesos que:

- se ejecutan en segundo plano
- no interactúan directamente con el usuario
- Prestan algún servicio al sistema o a otros programas.

Ejemplos: sshd, crond, systemd-journald.

Un **daemon** en *Linux es un programa que trabaja **en segundo plano**, sin que el usuario lo vea, prestando servicios al sistema. Su nombre viene de la palabra griega “daimon” que significa algo parecido a **espíritu o agente invisible que nos guía**, no a un demonio maligno.

Servicios vs *Daemon* - Linux

Cualquier proceso en segundo plano NO es un *daemon*,
pero todo *daemon* SÍ es un proceso en segundo plano

Un *daemon* normalmente: ofrece un servicio al sistema o a otros programas, está pensado para vivir independientemente de la terminal, suele arrancar al inicio o ser gestionado por *systemd* y sigue funcionando aunque cierres sesión, no está asociado a interacción directa con el usuario.

Ejemplos de *daemons* reales:

- sshd
- cron o crond
- cupsd
- systemd-journald



Servicios vs *Daemon* - Linux

Muchos **servicios** funcionan mediante un **daemon**. Muchas veces no se entiende uno sin el otro

- **servicio** → la forma en que el sistema gestiona el *daemon*.
- **daemon** → proceso de fondo que realiza la tarea o presta el servicio.

Ejemplo:

ssh.service → es el servicio gestionado por systemd que controla sshd
sshd → es el daemon

Servicios en Linux

systemctl list-units --type=service	Lista todas las unidades de tipo servicio.
systemctl status nombre_servicio	Devuelve el estado del servicio. Si está activo, inactivo o fallando
systemctl start nombre_servicio	Inicia el servicio
systemctl stop nombre_servicio	Detiene el servicio
systemctl restart nombre_servicio	Reinicia el servicio
systemctl enable nombre_servicio	Activa el arranque del servicio en el arranque del sistema
systemctl disable nombre_servicio	Desactiva el arranque del servicio en el arranque del sistema

Servicios en Linux

systemctl -- ejemplos con nginx

- **Listar servicios:** `systemctl list-units --type=service`
- **Ver solo fallidos:** `systemctl --failed`
- **Estado de uno:** `systemctl status nginx`
- **Ver si arranca al boot:** `systemctl is-enabled nginx`
- **Dependencias:** `systemctl list-dependencies nginx`
- **Unidades activas:** `systemctl list-units`

Servicios y Daemons

SERVICE WINDOWS

- En Windows
(**Gestionados por el sistema**)
 - Ejecutan tareas en segundo plano
 - Controlados vía **Services.msc**
- ➔ CLI: **\$ sc query, \$ net start/stop** para manejo por terminal



SERVICE LINUX

- En Linux (Gestión moderna systemd)
- Ejecutan tareas en **segundo plano**
- Controlados vía **systemctl**
- GUI: Inexistente por ser Linux minimalista
- CLI: **\$ systemctl start/stop/restart**



CLI: **\$ systemctl start/stop/restart**

DAEMON LINUX

- En Linux (Proceso en segundo plano típico de Unix/Linux)
- Ejecutados libres de UI, de largo plazo
- Son servicios persistentes que usa el sistema
- Suelen ejecutarse al arrancar el sistema



Como el daemon **sshd** (OpenSSH daemon): mantiene conexiones SSH.

Logs del sistema, servicios - journalctl

- Todo: **journalctl**
- In live: **journalctl -f (follow)**
- Arranque: **journalctl -b (boot)**
- Kernel: **journalctl -k**
- Por unidad: **journalctl -u nginx**
- Por proceso: **journalctl _PID=1234(servicio)**
- Por tiempo: **journalctl --since "2026-02-04 08:00" --until "09:00"**
journalctl --since "1 hour ago"
- Por prioridad: **journalctl -p warning**
- **Ejemplo**, logs del servicio nginx con follow: **journalctl -u nginx -f**

Logs del sistema

Logs del Kernel - dmesg

¡dmesg es el mensajero del kernel! Nos permite consultar el buffer de mensajes del kernel. Sería el primer sitio a consultar si nos falla algo de hardware, un usb que no se reconoce, problemas de memoria (OOM Killer), etc.

dmesg

dmesg -T → timestamps en formato humano

dmesg -w → Similar al -f de follor, se queda esperando nuevos mensajes.

dmesg -l err,warn → filtra solo errores y warnings.

Archivos de logs clásicos y fundamentales todavía /var/log/...

Podemos consultar los diferentes archivos de log directamente desde la carpeta

/var/log

- Arranque: /var/log/boot.log
- Gestor de paquetes:
 - APT: /var/log/apt/history.log, /var/log/apt/term.log
 - dpkg: /var/log/dpkg.log
- Login: /var/log/wtmp, /var/log/btmp, /var/log/lastlog

Consultar histórico de comandos del usuario actual

history

fichero ~/.bash_history

Visores de procesos GNU/Linux

- `ps ux`, `ps -u user` → usuario actual
- `ps aux`, `ps -ef` → all users
- `pstree` → ver procesos en vista de árbol
- `top` → ver procesos, cpu, RAM, en tiempo real.

Otros visores de procesos (requieren instalación):

- `bttop`
- `htop`
- `atop`

Automatización y programación de tareas en GNU/Linux

¿Qué es at?

at

at (atd)

Con **at** podremos programar la ejecución de comandos que se ejecutan **una sola vez** en el futuro. Surge para ejecutar tareas únicas, en plan “ejecuta este script a tal hora mañana y luego olvídate”.

Tiene una interfaz de *consola*, pero también se le pueden pasar parámetros con scripts o programas y/o los comandos vía pipes o con redirecciones.

Para la salir de la interfaz de **at** pulsaremos **CTRL+D** (modo interactivo) después de escribir los comandos que queremos se ejecuten a futuro.

at TIEMPO

at TIEMPO -f script.sh

at TIEMPO → modo interactivo

at TIEMPO -f script.sh → modo directo

EJEMPLOS	
at now +2 minutes	at 10:00 15.03.2026
at 3:30 pm	at tomorrow
at now + 1 hour	at tomorrow + 1 hour
at midnight	at 1:00 tomorrow
at noon, at teatime	at 4pm + 3 days
at now + 25 hours	at 02:00 + 1 week
at 13:00 + 1 month	at now + 1 year
at 5pm friday	at 0815am

at – sin modo interactivo

Ejemplos:

echo “ls -lh /home/user/ > /home/user/salida.txt 2>&1” | at now + 1 minute

at now + 2 minutes -f myscript.sh

→ RECOMENDADA

cat miscript.sh | at noon

→ NO RECOMENDADA

echo ‘bash /ruta/miscript.sh’ | at tomorrow

→ OK para forzar BASH y no SH.

atq, atrm y batch

atq → Listar todos los trabajos programados y pendientes con at. (at -l)

atrm ID → Eliminar trabajos pendientes por su ID (at -r ID). **Ejemplo:** atrm 5

at -c ID → Muestra todo el código/script interno que **at** ejecutará según los comandos dados.

batch → Se ejecutará cuando el sistema esté libre, tareas no urgentes. Equivale: at -q b -m now

atq, atrm y batch

EJERCICIOS

1. Creamos en modo interactivo un **at** que se ejecute en **2 min** desde el momento que hacemos el comando, y simplemente cree un archivo nuevo en nuestra carpeta de usuario:

at now +2 minute

➤ **touch ~/mi_fichero_con_at_vacio.txt**

➤ **CTRL+D**

atq → listamos las tareas

2. Ahora automatizamos un **ls** y el resultado lo guardamos en un archivo en el escritorio:

echo 'ls -lh > ~/Escritorio/RESULTADO.txt' | at now + 1 min

at now +1 min <<< "ls -lh > ~/Escritorio/RESULTADO.TXT"

atq, atrm y batch

EJERCICIOS

4. Nos mandamos un aviso al escritorio

```
echo 'notify-send "Holaaa desde la terminal!, este mensaje fue programado hace 2 minutos" ' | at now + 2 minutes
```

```
at now +2min <<< "notify-send 'Holaaa desde la terminal!, este mensaje fue programado hace par de minutos!!' "
```

5. Ahora combinamos varios comandos como sabemos ya un **ls** y el resultado lo guardamos en un archivo en el escritorio y mandamos también notificación:

```
at now + 1 minute <<< "mkdir -p ~/Escritorio/PRUEBA; ls -la /etc > ~/Escritorio/PRUEBA/listado.txt; notify-send 'Tarea completada' "
```

6. Aviso de que la clase ha terminado.

```
at 20:30 <<< "notify-send 'La clase ha terminado!! Nooo!' "
```

¿Qué es cron y crontab?

cron y crontab

Ya vimos que el comando **at** es muy útil para lanzar un *programa/comando/script* en un momento dado del futuro, **pero en cambio no nos sirve para tareas repetitivas.**

Es por eso que surge la necesidad de un programa similar, pero para tareas repetitivas en el tiempo, y justo eso es precisamente **cron**.

¡cron viene de Crono/Chronos de la mitología griega asociado al paso del tiempo! Y nació desde las versiones iniciales de Unix en los 70 allá en los laboratorios Bell.

cron y crontab

cron

Es un **daemon** que se inicia con el sistema y revisa cada minuto si hay alguna tarea programada para ejecutarse.

crontab (*cron table*)

Es el archivo de texto donde los usuarios guardan sus listas de tareas a ejecutar.

Esto nos permite automatizar backups, limpieza de logs, sincronización de bases de datos, generar recordatorios y un sinfín de tareas que un sistema operativo debe ejecutar de forma periódica sin que el administrador se tenga que preocupar.

NOTA: Cada usuario del sistema (incluido root) tiene su propio archivo crontab independiente.

La sintaxis de Crontab

MINUTO HORA DIA_MES MES DIA_SEMANA COMANDO

Campo	Valores posibles
Minuto	0-59
Hora	0-23
Día_Mes	1-31
Mes	1-12
Día_Semana	0-7 (0 y 7 siempre son domingo y 1 siempre es lunes)

NOTA: En cada campo se puede utilizar el * (asterisco) para indicar todos los valores.

La sintaxis de Crontab

Operadores especiales:

Operador	Significado
*	Cualquiera, siempre, todos los valores.
,	Lista de valores. Ej. 1,15,30 en el minuto 1, 15 y 30.
-	Rango de valores. Ej. 1-5 del 1 al 5 para días de la semana indicaría de Lunes a Viernes.
/	Incrementos de x valor. Ej. */15 en minutos significa “cada 15 minutos”.

¿Cada cuanto ejecutaría CRON el siguiente myscript.sh ?

* * * * * myscript.sh

cron y crontab

crontab

Es el comando de gestión que utilizaremos para editar y consultar los archivos *crontab* que tenemos como usuario:

crontab -e

→ **Edita**. Abre tu archivo crontab en el editor y te permite modificarlo. Es la forma natural de crear automatizaciones.

crontab -l

→ **Lista** por pantalla el contenido de tu crontab.

crontab -r

→ **Remove**. Borra todo tu archivo crontab.

crontab -u user -e

→ Permite a un *admin* editar el crontab de otro usuario.

cron y crontab

EJERCICIOS

```
* * * * echo "tic tac cron: $(date)" >> /home/dam/cronsito.log
```

```
*/5 * * * * echo "Heartbeat cada 5 min: $(date)" >> /tmp/heartbeat.log
```

```
30 8 * * 1 echo "Buenos días, es lunes! $(date)" > /home/dam/Escritorio/lunes.txt
```

“La memoria de at VS La amnesia de cron”



“La amnesia de *cron*”

El comando **cron** no “recuerda” casi nada del entorno interactivo que tenía el usuario en el momento de crear el *crontab* puesto que está pensado para ejecutar tareas repetitivas, predecibles, no interactivas y a lo largo de mucho tiempo. **cron se basa en ejecutar de forma limpia y repetible:**

- Ejecuta a la hora y fecha indicadas.
- Ejecuta con un **entorno mínimo**.
- No hereda ningún tipo de sesión interactiva.
- No existen ni aliases, ni funciones o variables que tenías en la terminal.
- obviamente el comando exacto a ejecutar.
- Conviene usar rutas absolutas y declarar variables necesarias.

***cron** es más como “cada día a esta hora ejecuta esto, pero empieza desde cero”*

“La memoria de *at*”

El comando **at** recuerda bastante del *contexto* desde el que fue lanzado. **at** genera un script con dicho contexto de forma muy precisa y es útil para algunas ejecuciones. En resumen, **at** conservará en su script:

- El usuario que programó el comando.
- El intérprete del shell de ejecución.
- Variables de entorno importantes
- Directorio de trabajo.
- La *umask*
- Y obviamente el comando exacto a ejecutar.

at recrea el concepto de “más tarde ejecuta esto como si siguiera estando justo aquí, de esta forma”

timers

Hoy en día se suelen utilizar mucho más para tareas precisas y más profesionales los **timers** como servicios de **systemd**



systemctl list-timers

anacron

anacron es una herramienta de planificación de **tareas periódicas en días**, no para horas exactas: su función es comprobar si un trabajo diario, semanal o mensual **no llegó a ejecutarse porque el equipo estaba apagado o no disponible**, y lanzarlo más tarde cuando el sistema vuelva a estar activo.

Hoy en día al instalar **anacron** en sistemas Debian-based modernos **anacron** sustituye a **cron** en los *daily, weekly, monthly*.

A diferencia de **cron**, no trabaja con expresiones tipo minuto/hora, sino con su propia configuración, normalmente en **/etc/anacrontab**, donde se define cada tarea indicando **cada cuántos días** debe ejecutarse, el **retraso** antes de lanzarla y el **comando** a ejecutar; por eso se usa sobre todo en portátiles o PCs que no están encendidos 24/7, para no perder mantenimientos periódicos como los *daily, weekly, monthly* de **cron**.

watch

watch sirve para repetir un comando **en tiempo real mientras tú estás delante de la terminal**, mostrando su salida actualizada cada pocos segundos, así que va bien para vigilar algo como `df -h`, `du -sh`, `ps`, `sensors` o un simple `ls -lh`

watch es para monitorización interactiva y temporal continua.

EJERCICIOS:

watch comando

watch “comandos”

watch -b -n SEGUNDOS “comadnos”



¡¡ SUERTE EN EL EXAMEN !!

Verán que es más sencillo de lo que parece

